



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Тамаш Пап

**Фрактал рендеровање са хибридном
паралелизацијом**
ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Рачунарство и аутоматика		
Студент:	Тамаш Пап	Број индекса:	РА 4/2022
Степен и врста студија:	Основне академске студије		
Област:	Рачунарски системи високих перформанси		
Ментор:	Др. Вељко Петровић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Фрактал рендеровање са хибридном паралелизацијом
--

ТЕКСТ ЗАДАТКА:

Изложити теоретске основе за, и онда имплементирати механизам који рачуна фрактале, нарочито сет Манделброта, користећи перформантне прорачуне на графичкој картици. Анализирати рад резултујућег решења.

Руководилац студијског програма:	Ментор рада:

Примерак за: О - Студента; О - Ментора
--



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1. Увод	1
1.1 Мотивација	1
1.2 Циљеви	1
2. Основе	3
2.1 Фрактали, општи појмови и примене	3
2.2 Претходни радови	4
2.3 Манделбровов скуп, дефиниција и својства	5
2.4 Основе рендеровања алгоритмом бекства	6
3. Алгоритми рендеровања	8
3.1 Основна скаларна варијанта алгоритма бекства	8
3.2 SIMD векторизација	9
3.3 Геометријски тестови раног изласка	11
3.4 Мариани-Силвер подела границе	13
3.5 Метод процене растојања	14
3.6 Теорија пертурбације	15
3.7 Апроксимација степеним редом	17
3.8 Нумеричка прецизност за дубоко увећање	18
3.9 Обилазак и локалност	19
4. Имплементација на графичком процесору	21
4.1 WebGPU кернели	21
4.2 CUDA позадинско решење	22
4.3 Обилазак на графичком процесору и распоред меморије	24
5. Хибридни распоређивач за процесор и графички процесор	26
5.1 Мотивација и дизајн	26
5.2 Декомпозиција плочица и класификација	27
5.3 Расподела посла и синхронизација	29
5.4 Адаптивно балансирање оптерећења помоћу PI регулатора	30
6. Експериментална методологија	33
6.1 Технолошки стек	33
6.1.1 Rust	33
6.1.2 GPU технологије	34
6.1.3 Технологија бенчмаркинга	34
6.2 Хардвер	35
6.3 Бенчмаркови и тест сцене	36
6.3.1 Фрактали, резолуције и бројеви итерација	36
6.3.2 Прикази	36
6.3.3 Серије увећања	37

6.3.4	Поређење између рендерера	37
6.4	Метрике	37
6.4.1	Време кадра и пропусност	37
6.4.2	Ефикасност израчунавања	38
6.4.3	Поређење контролисано по прецизности	38
6.4.4	Паралелно скалирање и енергија	38
6.4.5	Тачност и микроархитектонске метрике	38
7.	Резултати и дискусија	39
7.1	Основна варијанта на процесору и SIMD скалирање	39
7.2	Позадинска решења за графички процесор	41
7.3	Цевовод од почетка до краја	43
7.4	Хибридни CPU+GPU распоређивач	44
7.5	Теорија пертурбације	46
7.6	Поређење између пројеката	48
7.7	Енергетска ефикасност	50
7.8	Нумеричка валидација	50
8.	Закључак	52
	Биографија	56

1. УВОД

1.1 Мотивација

Фрактали су један од занимљивијих делова математике. Облик какав је Манделбровов скуп дефинисан је кратким, једноставним правилом, а ипак примена тог правила изнова и изнова производи границу неограничене детаљности [12]. Иста врста структуре јавља се и ван чисте математике, у обалским линијама, откуцајима срца, финансијским тржиштима, а има и практичну примену у компресији података, пројектовању мрежа и медицинској слици.

Рендеровање фрактала попут Манделбрововог скупа заснива се на алгоритму бекства (енг. *escape time algorithm*). За сваки пиксел извршава се кратка нумеричка петља све док вредност или премаши задату границу или се не достигне максималан број итерација. Израчунавање једног пиксела не зависи од резултата ниједног другог пиксела, тако да се читава слика, у принципу, може израчунати паралелно. У пракси, већина јавно доступног софтвера за рендеровање фрактала и даље обрађује пикселе један по један на једној нити процесора, остављајући преостала језгра процесора и графички процесор неискоришћеним. Софтвер који заиста користи графички процесор ретко истовремено користи и процесор. Када га ипак користи, то је обично само као резервна опција за машине без подржаног графичког процесора, а не као додатни извор рачунарске снаге.

Да ли је графички процесор заиста бржа опција није тако очигледно као што изгледа. Наиван, једнонитни код на процесору чини да убрзања захваљујући графичком процесору изгледају већа него што заиста јесу, па поштено поређење мора да пође од исправно оптимизоване основне варијанте на процесору [10]. Типичан лични рачунар већ поседује вишејезгарни процесор са SIMD инструкцијама и графички процесор, један поред другог. Коришћење само једног од њих траћи хардвер који корисник већ поседује. То је мотивација овог рада. Циљ је изградити рендерер који гура и путању процесора и путању графичког процесора онолико далеко колико свака од њих сама по себи може да иде. Затим се оне комбинују распоређивачем који дели посао између процесора и графичког процесора на основу тога који је од њих у датом тренутку заиста бржи за дати део слике [11]. Тиме се замењује уобичајени приступ у коме се „рендерер за процесор” и „рендерер за графички процесор” третирају као два одвојена програма.

1.2 Циљеви

Циљ овог рада је да рендерује Манделбровов скуп што је брже могуће на једном личном рачунару, користећи процесор и графички процесор заједно, а не само један од њих. Достицање тог циља подразумева рад на неколико мањих циљева, од којих је сваки обрађен у некој од наредних глава.

Први је сам рендерер за процесор. Полазећи од скаларне основне варијанте алгоритма бекства, рад додаје вишејезгарну паралелизацију, SIMD (f64x4, f32x8), као и геометријске тестове раног изласка, кардиоидну проверу и проверу периода 2 и периода 3, који прескачу петљу итерације за пикселе за које се већ зна да припадају скупу. Поред тога, метод процене растојања и Мариани-Силвер (енг. *Mariani-Silver*) подела

границе смањују непотребан рад итерирања тако што попуњавају велике униформне области слике без итерирања сваког пиксела у њима. Плочичасти обилазак поређан по Хилбертовим или Муртоновим кривама додатно побољшава локалност приступа меморији. Посебан проблем представља дубоко увећање. Обична аритметика двоструке тачности (енг. *double*) исцрпи своју прецизност далеко пре него што се достигне занимљив ниво увећања. Рендерер стога захтева и теорију пертурбације, са једном референтном орбитом или са више референтних орбита уз ребазирање, апроксимацију степеним редом и *double-double* аритметику, како би остао тачан и брз [7].

Други циљ односи се на страну графичког процесора. То подразумева имплементацију истих кернела алгоритма бекства као шејдера за прорачунавање графичког процесора, једном преносиво, помоћу *wgpi* и *WebGPU*, и једном као ручно оптимизовано *CUDA* позадинско решење. Свака имплементација користи редослед обиласка и распоред меморије прилагођен графичком процесору, уместо директног преноса кода са процесора.

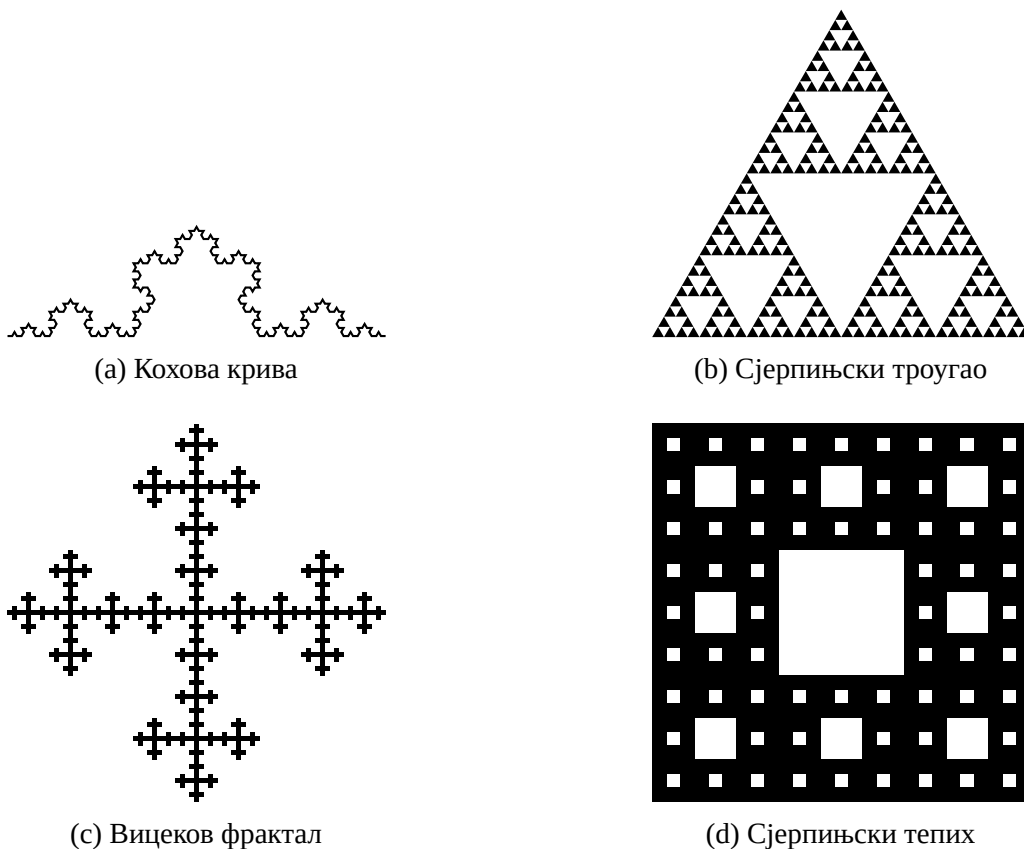
Трећи циљ повезује претходна два. Хибридни распоређивач за процесор и графички процесор дели кадар на плочице, класификује сваку плочицу по очекиваној цени и истовремено распоређује плочице по радним нитима процесора и графичком процесору. Затим у реалном времену прилагођава расподелу помоћу *PI* регулатора повратне спреге, вођеног измереном пропусношћу сваког уређаја.

Четврти циљ је да се све ово правилно измери, пошто ништа од претходног нема велику вредност без доказа. То значи дефинисање окружења за мерење: хардвера, сцена за бенчмарк на плитком и дубоком увећању, као и метрика пропусности и кашњења. Затим значи мерење стварног доприноса сваке оптимизације и поређење конфигурација само-процесор, само-графички-процесор и хибрид међусобно, као и у односу на постојеће рендерере фрактала отвореног кода. Ово последње поређење је важно само по себи. Хибридни распоређивач вреди своје додатне сложености само ако мерљиво надмашује одвојено покретање рендерера за процесор и рендерера за графички процесор.

2. ОСНОВЕ

2.1 Фрактали, општи појмови и примене

Пре него што се пажња усмери конкретно на Манделбровтов скуп, вредно је рећи шта је фрактал уопште. Фрактал је облик чија се сложеност не губи увећањем. Круг, што се пажљивије посматра, све више личи на праву линију, али фрактал на сваком нивоу увећања и даље показује структуру, а његови мали делови личе на цео облик, било потпуно тачно, било у статистичком смислу [12]. Кохова крива, Сјерпињски троугао, Вицеков фрактал и Сјерпињски тепих, приказани на слици 2.1.1, четири су класична примера. Сваки од њих настаје понављањем једноставног правила замене над почетним обликом, изнова и изнова, без краја.



Слика 2.1.1: Четири фрактала настала понављањем једноставног правила замене над почетним обликом. Сва четири приказана су након коначно много корака. Прави фрактал представља границу овог процеса понављањем бесконачно.

Оваква поновљена конструкција фракталима даје њихово најкарактеристичније математичко својство: димензију која није цео број, названу Хаусдорфова димензија. За облик састављен од N примерака самог себе, при чему је сваки умањен фактором r , димензија сличности износи

$$D = \frac{\log N}{\log(1/r)}. \quad (2.1.1)$$

Кохова крива сваку дуж замењује са $N = 4$ дела размере $r = 1/3$, што даје $D = \log 4 / \log 3 \approx 1.26$. Сјерпињски троугао користи $N = 3$ дела размере $r = 1/2$, што даје $D = \log 3 / \log 2 \approx 1.58$. Вицеков фрактал задржава средишње и четири суседна ивична поља мреже 3×3 ($N = 5$, $r = 1/3$), што даје $D \approx 1.46$, а Сјерпињски тепих задржава осам спољашњих поља исте мреже и одбацује средишње ($N = 8$, $r = 1/3$), што даје $D \approx 1.89$. Све четири вредности налазе се строго између 1 и 2. Ови облици су грубљи од линије, а ипак се никада не задебљају у попуњену површину, и управо тај јаз између фракталне димензије облика и његове уобичајене тополошке димензије је, неформално речено, оно што облик уопште чини фракталом [5]. Мало природно насталих фрактала настаје по тачном правилу понављања попут ова четири. Већина њих, укључујући и Манделбровов скуп, самослична је само у лабавијем, статистичком смислу, а не тачном, разлика на коју се одељак 2.3 касније враћа.

Фрактална геометрија није само математичка занимљивост. Манделбровова полазна тачка била је практично запажање: измерена дужина обалске линије расте што се прецизније мери, јер се обала понаша као фрактална крива, а не глатка. Уобичајена еуклидска геометрија нема добар начин да то опише [12]. Иста врста гранајуће, самосличне структуре јавља се широм природе из функционалног разлога, а не случајно. Речне мреже, планински венци и облачне границе имају то својство, као и људско тело, где се плућа и крвни судови гранају фрактално како би сместили велику површину размене у малу запремину. Практичне примене директно следе из ових запажања. Барнслијев оквир итерираних функцијских система (енг. *iterated function system*) иста је математика која стоји иза самосличних облика попут Сјерпињског троугла. Он је такође био основа за рану шему компресије фракталних слика која слику чува као мали скуп трансформација, а не као сирове пикселе [1]. Фрактални дизајн антена сажима самосличну геометрију у мали физички простор како би резонирала на неколико фреквенција истовремено, због чега се такве антене налазе у компактним мобилним уређајима. Рачунарска графика користи фрактални шум за процедурално генерисање терена, облака и текстура уместо ручне израде. Фрактална димензија користи се и као дијагностичка мера у медицини, где се неправилност ткива или крвних судова, изражена као фрактална димензија, мерљиво разликује између здравог и оболелог ткива. Манделбровов скуп, конкретан фрактал који овај рад рендерује, припада истој широј породици, али настаје из динамичког система, а не из фиксног геометријског правила замене.

2.2 Претходни радови

Постојећи рендерери фрактала грубо се деле у три групе, у зависности од тога колико од доступног хардвера заиста користе.

Прва група рендерује искључиво на процесору, а брзину постиже паметним алгоритмима, а не додатним хардвером. Ова група такође обухвата четири рендерера са којима се овај рад директно пореди у глави 7. ХаоС [21] је интерактивни увеличавач у реалном времену који остаје течан тако што поново користи пикселе израчунате за претходни кадар и рачуна само оно што се променило, уместо да сваки кадар рачуна све пикселе изнова. Данас ради и као веб апликација, али и даље без коришћења графичког процесора за саму фракталну математику. FractalNow [18] је вишенитан и додаје хеуристике попут адаптивног анти-алијасинга, али такође задржава сво израчунавање на процесору. FractalRendererC++ [20] је C++ рендерер са интерактивном навигацијом мишем и тастатуром, а Fractals-rs [3] је Rust истраживач фрактала са SIMD подршком

на процесору. Сва четири су рендерери са једном позадином, искључиво на процесору, без пута ка графичком процесору, што их чини корисном основом за то колико далеко може да се стигне само оптимизацијом на страни процесора. Пети рендерер искључиво на процесору, Fractive [14], није део тог скупа за поређење, али заслужује да се посебно помене. Он већ векторизује свој кернел на процесору помоћу SSE2, поред вишејезгарне паралелизације, што је управо врста оптимизације на страни процесора коју овај рад такође примењује. Његова употреба OpenGL-а, ипак, ограничена је на приказ 3Д мапе висина већ израчунате слике, а не на израчунавање самог фрактала.

Друга група циља на дубоко увећање, а не на сирову брзину, и по нужности је углавном ограничена процесором. Kalles Fraktaler и његов огранак Nanobrot [6] имплементирају теорију пертурбације. Једна референтна орбита високе прецизности рачуна се једном по кадру, а сваки пиксел се затим рендерује у односу на ту референцу помоћу јефтине аритметике двоструке тачности, што уопште омогућава практичну употребу нивоа увећања далеко изван обичне двоструке тачности [7]. Ова нумеричка техника је ортогонална паралелизацији, а одељак 3.6 се на њу директно враћа за рендерер изграђен у овом раду.

Изван рендеровања фрактала, општи проблем поделе податковно-паралелног посла између процесора и графичког процесора има сопствену литературу. Ли и др. [10] показују да су тада уобичајене тврдње да је „графички процесор до 100 пута бржи” углавном биле последица поређења оптимизованог кода за графички процесор са неоптимизованим, једнонитним кодом за процесор. Они такође показују да правилно подешена имплементација на процесору затвара већи део те разлике. Ово је аргумент коришћен у одељку 1.1 за то зашто је страна процесора у овом раду захтевала стварну оптимизацију пре него што било какво поређење процесора и графичког процесора може бити поуздано. Qilin [11] се бави комплементарним проблемом: како поделити посао између процесора и графичког процесора када су оба довољно брза. У реалном времену профилише мали узорак посла на сваком уређају и адаптивно распоређује остатак на основу измерене пропусности, иста општа идеја коју глава 5 примењује конкретно на плочице фракталне слике.

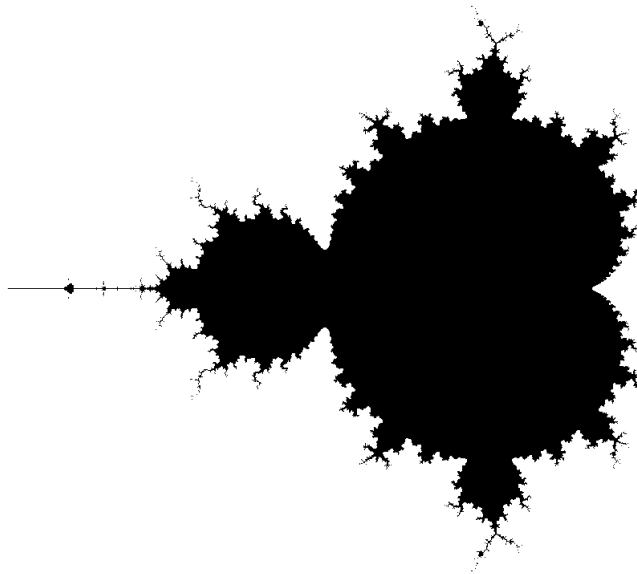
Ниједан од горе прегледаних рендерера не комбинује SIMD-оптимизовано рендеровање на процесору, позадинско решење израчунавања на графичком процесору и распоређивач вођен повратном спрегом у реалном времену између њих на нивоу плочица пиксела. Управо ту комбинацију остатак овог рада гради и вреднује.

2.3 Манделбровов скуп, дефиниција и својства

За комплексан број c , посматрајмо низ добијен итерирањем квадратног пресликавања

$$z_0 = 0, \quad z_{n+1} = z_n^2 + c. \quad (2.3.1)$$

Манделбровов скуп M је скуп свих $c \in \mathbb{C}$ за које овај низ остаје ограничен, односно не тежи бесконачности када $n \rightarrow \infty$ [12]. Свака тачка комплексне равни или припада M или му не припада, и управо тај једноставан тест припадности, примењен на сваки пиксел слике, заправо производи слику скупа. Детаљи тог теста су тема одељка 2.4.



Слика 2.3.1: Манделбровов скуп над $c_r \in [-2.5, 1.0]$, $c_i \in [-1.25, 1.25]$

M поседује неколико својстава која су директно важна за његово рендеровање, сва видљива на слици 2.3.1. Прво, ограничен је. Ако је $|c| > 2$, низ из једначине 2.3.1 гарантовано дивергира, тако да M у целости лежи унутар диска $|c| \leq 2$. Ово следи јер, чим постане $|z_n| > 2$ и $|z_n| \geq |c|$, сваки наредни корак удаљеност од координатног почетка умножава фактором већим од 1, тако да $|z_n|$ неограничено расте. Исти аргумент оправдава рано заустављање итерације за било који пиксел чим $|z_n|$ пређе тај праг, уместо бесконачног итерирања. Друго, M је повезан. Упркос томе што визуелно делује као скуп неповезаних „острва”, свака тачка скупа може се повезати са сваком другом тачком путем који остаје унутар M , што су Дуади и Хабард доказали кроз униформизацију скупа спољашњим зрацима. Треће, M је самосличан у лабавом смислу. Његова граница садржи бесконачно много малих, изобличених копија целог скупа, на сваком нивоу увећања, познатих као сателитске или „мини-Манделброт” копије. Управо због тога увећавање границе стално открива нови детаљ, уместо да конвергира ка једноставној кривој. То је такође разлог зашто је рендеровање дубоког увећања, одељак 3.6, уопште занимљив проблем, а не само рачунање исте слике на мањој размери. Најзад, иако је M дводимензиона област равни, његова граница је фрактална крива Хаусдорфове димензије тачно 2. То што је крива толико „испуњавајућа” у простору је, неформално речено, разлог зашто је тест бекства близу границе нумерички деликатан и зашто су пиксели чија класификација траје најдуже концентрисани управо тамо где слика има највише визуелног детаља.

2.4 Основе рендеровања алгоритмом бекства

Једначина 2.3.1 се на стварном рачунару не може итерирати бесконачно, па рендерер мора тачан тест припадности да претвори у две практичне апроксимације.

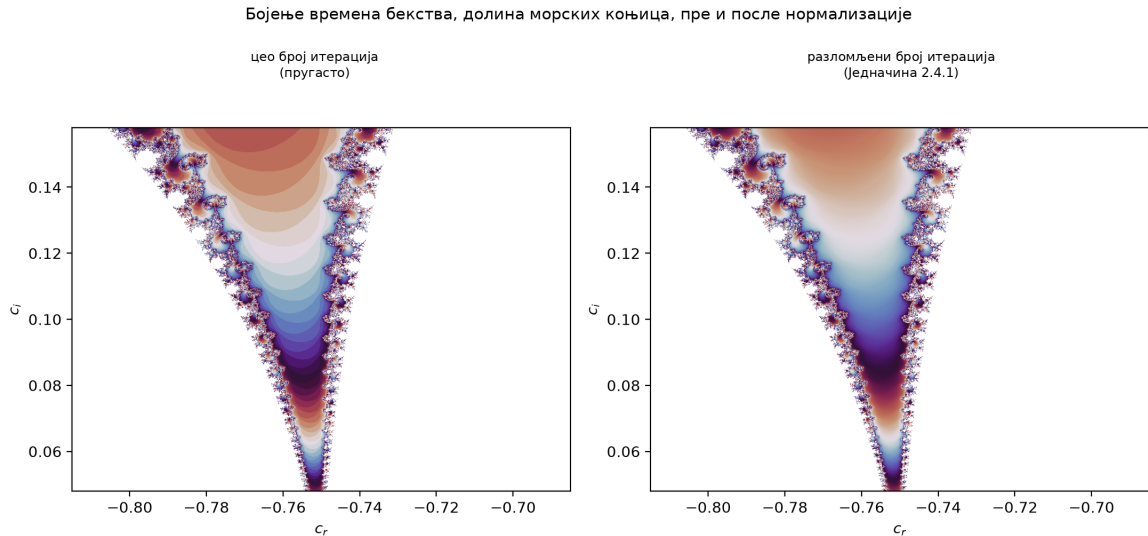
Прва је праг излаза (енг. *bailout*). Уместо да се чека да $|z_n| \rightarrow \infty$, итерација стаје чим $|z_n|$ пређе фиксни радијус бекства. Радијус од 2 већ је довољан да се гарантује да ће тачка дивергирати, истим аргументом коришћеним у одељку 2.3, па је сваки већи радијус безбедан, мада понекад непотребан, избор. Рендерери који желе глађе бојење често намерно користе већи радијус бекства, по цену неколико додатних итерација по

пикселу који побегне. Друга апроксимација је максималан број итерација. Пиксел који не побегне до достизања тог ограничења третира се као да припада скупу, иако би нешто виши лимит на крају можда показао да и он бежи. Подизање тог ограничења чини слику тачнијом, посебно близу границе и на дубоком увећању, али и сваки пиксел који никада не побегне, типично већина слике при великом увећању, чини сразмерно скупљим за рендеровање. Број извршених итерација стога је веома неуједначен по слици. Пиксели далеко од M бегају готово тренутно, пиксели дубоко унутар скупа извршавају пуно ограничење итерација, а пиксели близу границе могу непредвидиво учинити и једно и друго. Ова неуравнотеженост оптерећења главни је разлог зашто наивна паралелизација по пикселу лоше ради, и то је основни проблем који оптимизације у глави 3 и распоређивач у глави 5 заобилазе.

Претварање сировог броја итерација у слику такође захтева пажњу. Директно мапирање целобројног броја итерација у боју ствара видљиво пругање, оштре прстенове тамо где се број мења за један, уместо глатког прелаза. Стандардно решење је глатко, или „нормализовано”, бојење, које користи вредност z_n у тренутку бекства за израчунавање разломљеног броја итерација

$$\nu = n + 1 - \frac{\log \log |z_n|}{\log 2}, \quad (2.4.1)$$

тако да суседни пиксели који су побегли, рецимо, након 41,3 и 41,7 итерација добију видљиво различите, интерполиране боје, уместо да буду сврстани у исту траку. Опсег вредности ν присутних у кадру потпуно се мења са нивоом увећања и локацијом. Добро мапирање ν у палету боја обично такође захтева корак нормализације, на пример изједначавање хистограма. Тиме се обезбеђује да се доступни унос палете троше тамо где слика заиста има детаље, а не на бројеве итерација који се у том конкретном кадру једва јављају.



Слика 2.4.1: Иста гранична област обојена сировим целобројним бројем итерација и глатким бројем из једначине 2.4.1. Прстенови на левој слици настају понављањем палете једном по целој итерацији. Десна слика пролази кроз исту палету, али интерполира унутар сваког прстена.

3. АЛГОРИТМИ РЕНДЕРОВАЊА

3.1 Основна скаларна варијанта алгоритма бекства

Основна варијанта рендерера директна је, невекторизована имплементација итерације из једначине 2.3.1: један комплексан број типа `f64` по пикселу, један пиксел у једном тренутку. Две ствари је разликују од уџбеничке петље, и обе су важне за то шта „основна варијанта” значи у остатку ове главе. Прво, пре него што уопште почне итерирање, извршавају се аналитички тестови раног изласка из одељка 3.3, тако да основна варијанта већ искључује два највећа унутрашња дела скупа из петље итерације. Свака касније описана оптимизација у овој глави мери се у односу на основну варијанту која већ садржи овај тест, а не у односу на наивну петљу без икаквих пречица. Друго, корак ажурирања $z_{n+1} = z_n^2 + c$ записан је као једна спојена инструкција множења и сабирања (енг. *fused multiply-add*), а не као одвојено множење и сабирање, тако да хардвер извршава један корак заокруживања уместо два.

Сам тест бекства користи радијус успостављен у одељку 2.4, $|z_n|^2 > 4$, проверен као поређење квадрата како би се избегло вађење квадратног корена у свакој итерацији. Поред прага излаза, рендерер извршава детекцију циклуса у Брентовом стилу [2]. На сваких неколико итерација чува тренутну вредност z_n , а у каснијим итерацијама је пореди са том сачуваном вредношћу. Ако се орбита врати на удаљеност мању од 10^{-20} од ње, тачка се сматра периодичном и одмах се класификује као да припада скупу, без чекања на `max_iter`. Интервал поређења почиње на 8 итерација и удвостручује се после сваке провере, до горње границе од 512. На овај начин провера се често извршава док постоји велика шанса да се исплати рано, а ретко касније, када је мало вероватно да ће се активирати и када би иначе само додавала непотребан трошак по итерацији. Ово хвата циклусе које кардиоидни и балонски тестови пропуштају. Ти тестови су затворени облик за два највећа дела скупа. Граница, међутим, садржи бесконачно много мањих периодичних делова, видљивих мањих балона и ситних сателитских копија из одељка 2.3, које Брентов метод и даље може генерички да ухвати, без унапред познатог облика.

Посао се дели по језгрима процесора у податковно-паралелним блоковима редова, а сама петља по пикселима не садржи ниједно дељење или тригонометријску операцију по пикселу. Пресликавање координата пиксела у комплексан број своди се, једном по кадру, на четири константе, почетак и корак за реални и имагинарни део, тако да је координата сваког пиксела удаљена један корак спој-множи-сабери од претходне. На референтној машини коришћеној кроз цео овај рад, глава 6, ова основна варијанта одржава пропусност од отприлике 195 до 239 милиона пиксела у секунди, за резолуције од 800×600 до 3840×2160 , за Манделбровов скуп при 1000 итерација. Пропусност је практично константна у односу на резолуцију, што се и очекује за посао чија цена по пикселу не зависи од величине слике око њега.

Листинг 3.1.1 показује ово језгро: позив теста затвореног облика на самом почетку, а затим петљу итерације са Брентовом детекцијом циклуса, описаном текстуално изнад.

```

pub(crate) fn mandelbrot(cr: f64, ci: f64, max_iter: u32) -> f32 {
    if in_cardioid_or_period2(cr, ci) || in_period3_bulb(cr, ci) {
        return max_iter as f32;
    }

    let mut zr = cr;
    let mut zi = ci;
    let (mut zr_b, mut zi_b) = (0.0, 0.0);
    let mut period = 0u32;
    let mut check = 8u32;

    for i in 2..max_iter {
        let zr2 = zr * zr;
        let zi2 = zi * zi;
        let zn_sq = zr2 + zi2;
        if zn_sq > ESCAPE_RADIUS_SQ {
            return smooth_iter(i, zn_sq, max_iter);
        }
        let new_zi = (2.0 * zr).mul_add(zi, ci);
        zr = zr2 - zi2 + cr;
        zi = new_zi;

        let dr = zr - zr_b;
        let di = zi - zi_b;
        if dr * dr + di * di < 1e-20 {
            return max_iter as f32;
        }
        period += 1;
        if period == check {
            period = 0;
            check = check.saturating_mul(2).min(512);
            zr_b = zr;
            zi_b = zi;
        }
    }
    max_iter as f32
}

```

Листинг 3.1.1: Скаларна f64 итерација Манделбрововог скупа, референтна имплементација из које су SIMD, CUDA и WGSL кернели алгебарски изведени (src/fractal/kernels/mandelbrot.rs).

3.2 SIMD векторизација

Скаларна петља из одељка 3.1 рачуна итерацију једног пиксела у једном тренутку, иако модерни процесори могу применити исту аритметичку операцију на неколико вредности у једној инструкцији. SIMD кернели рендерера замењују скаларно ажурирање $z_{n+1} = z_n^2 + c$ истим ажурирањем примењеним на неколико пиксела истовремено, спакованих у један векторски регистар: f64x4 са четири f64 траке (енг. *lanes*) и f32x8 са осам f32 трака. Који ће се користити зависи од дубине увећања, преко истог прага од 10^6 који се кроз цео овај рад користи за избор између f32 и f64. Испод те границе рендерер користи шири и јефтинији f32x8 кернел. Изнад ње, приближно седам децималних цифара прецизности које нуди f32 више нису довољне да разликују суседне пикселе, па се прелази на f64x4.

Векторизација петље алгоритма бекства уводи проблем ког скаларна петља нема. Различите траке беже при различитом броју итерација, али SIMD инструкција примењује исту операцију на све траке истовремено, тако да ниједна трака сама по себи не може рано да стане. Кернели ово решавају маском активних трака, бит-узорком по траци, свих јединица или свих нула, која се са ажурирањем комбинује бит-по-бит операцијом И (AND). Чим $|z_n|^2$ одређене траке пређе радијус бекства, бележе се њена итерација бекства и вредност z_n , а маска ту траку искључује, тако да даља ажурирања за њу постају без ефекта, док остале траке настављају. Петља стаје тек када све траке побегну или се достигне `max_iter`.

Трећи кернел, коришћен само за Манделбровов скуп, иде корак даље тако што покреће два независна ланца `f32x8` упоредо, шеснаест пиксела по кораку уместо осам. Два ланца су иначе идентична. Поента је паралелизам на нивоу инструкција, а не додатна ширина вектора. Модерни процесори могу имати неколико спојених инструкција множења и сабирања у лету истовремено, ако постоји други, независан ланац посла који попуњава цевовод док се резултат првог ланца још увек рачуна. Измерено на резолуцији 1920×1080 за Манделбровов скуп при 1000 итерација, кернел са два ланца рендерује кадар за 4.25 ms, наспрам 4.62 ms за једноструки `f32x8` кернел, 6.04 ms за `f64x4` и 8.17 ms за скаларну основну варијанту, што је убрзање од $1.92\times$ у односу на скаларну варијанту, од чега само око 8% потиче од самог другог ланца. То што је додатни добитак од паралелизма на нивоу инструкција мали није изненађење када се узме у обзир одељак 3.3. Кардиоидни и балонски тестови већ уклањају велики део унутрашњих пиксела пре него што „врџа” петља уопште крене, остављајући мање кашњења које би други ланац могао да сакрије. Једно мерење током бенчмаркинга такође је показало да се два наизглед идентична, поновљена мерења истог кернела разликују за више од половине. Глава 6 се на ово враћа као подсетник да је једно мерење бенчмарка само шум све док се не понови.

Листинг 3.2.1 показује маску `active` из `f32x8` кернела описану изнад: провера `just_escaped` бележи која поља су управо побегла бит-по-бит операцијом И, а ажурирање `active` одмах затим искључује та поља из маске тако да их даља ажурирања више не дотичу, иако цео вектор наставља да се рачуна до краја петље.

```

let mut active = f32x8::from([
    f32::from_bits(if in_set[0] { 0 } else { u32::MAX })),
    f32::from_bits(if in_set[1] { 0 } else { u32::MAX })),
    f32::from_bits(if in_set[2] { 0 } else { u32::MAX })),
    f32::from_bits(if in_set[3] { 0 } else { u32::MAX })),
    f32::from_bits(if in_set[4] { 0 } else { u32::MAX })),
    f32::from_bits(if in_set[5] { 0 } else { u32::MAX })),
    f32::from_bits(if in_set[6] { 0 } else { u32::MAX })),
    f32::from_bits(if in_set[7] { 0 } else { u32::MAX })),
]);
let mut escape_iter = [max_iter; 8];
let mut escape_zn    = [0.0f32; 8];

for i in 2..max_iter {
    let zr2 = zr * zr;
    let zi2 = zi * zi;
    let zn_sq = zr2 + zi2;

    let just_escaped = zn_sq.cmp_gt(escape) & active;
    if just_escaped.any() {
        let mask: [f32; 8] = just_escaped.into();
        let zn: [f32; 8] = zn_sq.into();
        for lane in 0..8 {
            if mask[lane].to_bits() != 0 {
                escape_iter[lane] = i;
                escape_zn[lane] = zn[lane];
            }
        }
        active = active & (all_one ^ just_escaped);
        if !active.any() { break; }
    }

    zi = (two * zr).mul_add(zi, ci);
    zr = zr2 - zi2 + cr;
}

```

Листинг 3.2.1: Маска активних трака у f32x8 Манделбровом кернелу (src/fractal/kernels/mandelbrot.rs).

3.3 Геометријски тестови раног изласка

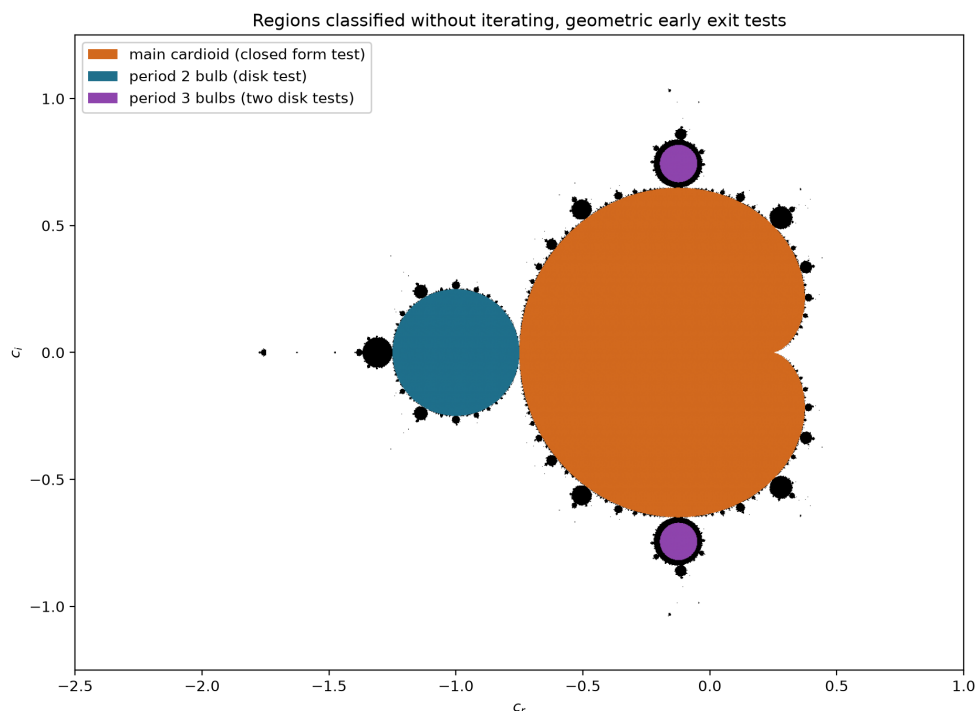
Две области Манделбрововог скупа довољно су велике и довољно једноставног облика да се припадност може директно тестирати из c , без икаквог итерирања, стандардно убрзање описано у [19]. Главна кардиоида, највећа компонента, представља слику јединичне кружнице под пресликавањем $c = \frac{1}{2}e^{i\theta} - \frac{1}{4}e^{2i\theta}$. Еквивалентно, уз ознаку $q = (c_r - 0.25)^2 + c_i^2$, тачка припада кардиоиди тачно када

$$q(q + c_r - 0.25) < 0.25 c_i^2. \quad (3.3.1)$$

Балон периода 2, велика кружна компонента прикачена на леву страну кардиоиде, тачан је диск, $(c_r + 1)^2 + c_i^2 < 0.0625$, полупречника 0.25 са центром у $c = -1$. Оба теста су тачни алгебарски идентитети, а не апроксимације, тако да свака тачка коју они класификују као унутрашњу то заиста и јесте, и може јој се директно доделити `max_iter`, без икакве итерације.

Рендерер такође тестира два балона периода 3, следеће по величини унутрашње компоненте, прикачене на кардиоиду изнад и испод балона периода 2. Они немају тако чист затворени облик као кардиоида, али је сваки од њих ипак добро апроксимиран диском, са центром близу $c \approx (-0.122561, \pm 0.744862)$ и полупречником ≈ 0.073715 . Поређење квадрата растојања са квадратом полупречника поново избегава вађење квадратног корена. После периода 3, граница наставља без краја да производи мање балоне и сателитске копије, одељак 2.3, па више нема смисла ручно изводити затворени тест за сваки од њих. Мање периодичне компоненте уместо тога хвата генерички механизам детекције циклуса, већ уграђен у основну варијанту, одељак 3.1.

Ова три теста извршавају се прво, пре петље итерације, за сваки пиксел у сваком кернелу у овој глави, скаларном и SIMD подједнако. Тестови за кардиоиду и период 2 директно се векторизују, пошто су то само аритметичке операције над SIMD тракама. Тест за период 3, чији се центри дискова не векторизују тако чисто, уместо тога се процењује по траци у скаларној прецизности, и то само за пикселе које прва два теста нису већ разрешила. Главна кардиоида и балон периода 2 два су највеће области скупа по површини. Само овај тест стога обухвата велики удео пиксела који би иначе трчали до `max_iter` не радећи ништа корисно, управо оних „тешких” пиксела идентификованих као главни извор неуравнотежености оптерећења у одељку 2.4.

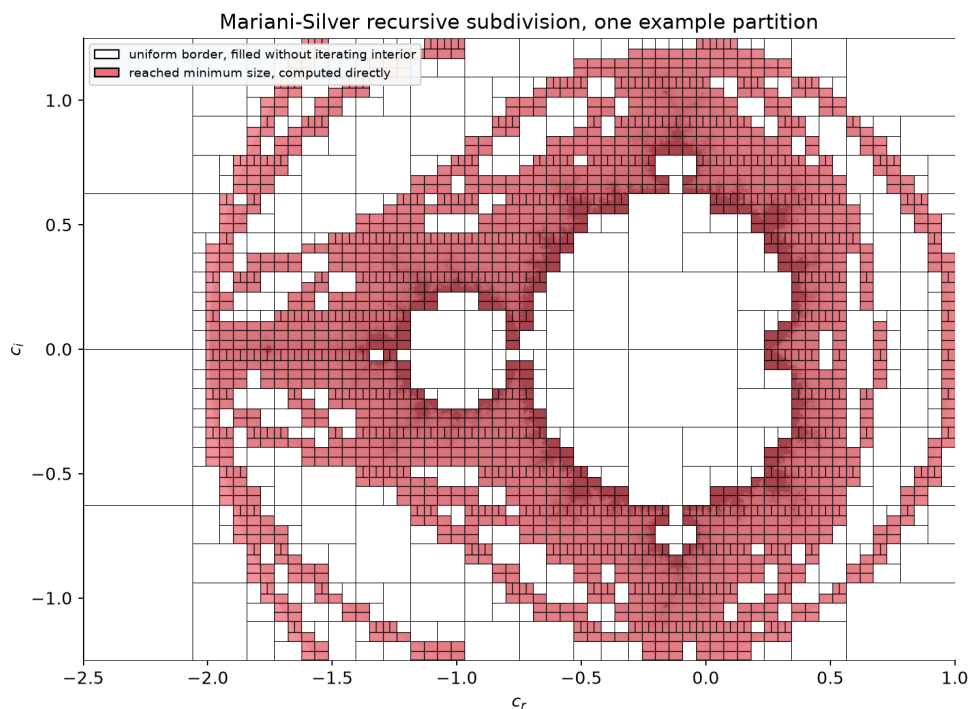


Слика 3.3.1: Три области затвореног облика приказане на рендеру целог кадра Манделбрововог скупа: главна кардиоида, балон периода 2 и два балона периода 3. Сваком пикселу унутар осенчене области директно се додељује `max_iter`, без икакве итерације. Свуда другде, укључујући и мање сателитске балоне видљиве дуж границе, и даље се извршава пуна петља.

3.4 Мариани-Силвер подела границе

Тестови из одељка 3.3 у затвореном облику класификују појединачне пикселе као унутрашње. Мариани-Силвер подела [19] уместо тога користи чињеницу да велике области *спољашњости* скупа, далеко од границе, такође деле исти број итерација бекства са својим суседима. Алгоритам затим покушава да открије и попуни такве области без израчунавања сваког пиксела унутар њих.

За дати правоугаоник слике, алгоритам рендерује само његову ивицу: горњи и доњи ред и леву и десну колону. Ако сви пиксели ивице имају исту вредност, правоугаоник се сматра униформним и цела његова унутрашњост попуњава се том вредношћу, без икаквог додиривања пиксела унутар ње. Ако ивица није униформна, правоугаоник је премали да би та претпоставка била безбедна. Уместо тога, дели се на два дела дуж своје дуже осе, тако да оба дела остану ближа квадрату него да настану дуге, издужене траке, и исти тест се рекурзивно примењује на сваки од њих. Ово се наставља све до минималне величине од 2×2 пиксела, испод које подела више не исплати сама себе, па се преостали пиксели једноставно рачунају директно. Суседни правоугаоници деле ивицу, па се бафер кадра унапред попуни сигналном (енг. *sentinel*) вредношћу, а пиксел ивице који је већ израчунат од стране суседног правоугаоника никада се не рачуна поново.



Слика 3.4.1: Пример поделе приказан на рендеру целог кадра Манделбрововог скупа; минимална величина овде је ради видљивости подигнута на 6×6 пиксела, док сам рендерер стаје на 2×2 . Бели правоугаоници имали су униформну ивицу и попуњени су без додиривања своје унутрашњости. Црвени правоугаоници достигли су минималну величину и израчунати су директно.

Претпоставка о униформној ивици није строго загарантована. У принципу, таканит пиксела различите вредности могао би да прође кроз унутрашњост правоугао-

ника, а да никада не додирне његову ивицу, у ком случају би корак попуњавања преко њега прешао бојом. У пракси је ово довољно ретко, а визуелна цена довољно мала, да алгоритам ово прихвата као познату, ограничену апроксимацију ризика, уместо да се експлицитно штити од ње.

Друга варијанта укључује и процену растојања из одељка 3.5. Уместо да само проверава да ли је ивица униформна, она такође проверава да ли је свако теме правоугаоника даље од границе скупа него што износи сопствена дијагонала правоугаоника, скалирана подесивим сигурносним фактором k . Ако јесте, зна се да је цео правоугаоник довољно далеко од сваког граничног детаља да је билинеарна интерполација између четири вредности у теменима визуелно неразлучива од израчунавања сваког пиксела. Правоугаоник се тада тако попуњава, уз проверу исправности да се глатки бројеви итерација у теменима не разликују за више од 2.0. Ова провера је важна јер би билинеарно попуњавање преко области у којој се основна функција брзо мења видљиво погрешно рендеровало ту област.

3.5 Метод процене растојања

Метод процене растојања [19] одговара на другачије питање од теста бекства: не колико итерација треба да c побегне, већ колико је c удаљено од границе скупа. Строго математичко оправдање ове величине долази из теорије потенцијала спољашњости скупа [15]. Процена следи из праћења, паралелно са обичном орбитом $z_{n+1} = z_n^2 + c$, њеног извода по c .

$$z_0 = 0, \quad \frac{dz_0}{dc} = 0, \quad z_{n+1} = z_n^2 + c, \quad \frac{dz_{n+1}}{dc} = 2z_n \frac{dz_n}{dc} + 1, \quad (3.5.1)$$

добитијеног диференцирањем саме итерације члан по члан. Ако орбита побегне у итерацији n , растојање од c до најближе тачке границе приближно износи

$$d \approx \frac{|z_n| \ln |z_n|}{|dz_n/dc|}. \quad (3.5.2)$$

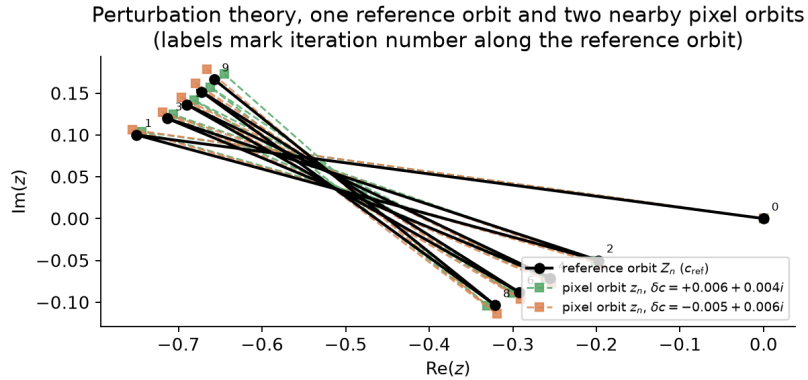
Ово је стандардан резултат за процену спољашњег растојања код фрактала заснованог на алгоритму бекства, не нешто специфично за овај рендерер. Оно што јесте специфично за овај рендерер јесте начин на који се та вредност користи. Уместо да покреће посебан режим анти-алијасинга или контурног рендеровања, d се користи тачно једном: као тест одсецања унутар DEM-проширене Мариани-Силвер варијанте из одељка 3.4. За правоугаоник чија су сва четири темена даља од границе него што износи сопствена дијагонала правоугаоника гарантовано је да не садржи ниједан фини гранични детаљ, па се безбедно може попуњити интерполацијом уместо итерирањем пиксел по пиксел. Пошто се dz_n/dc акумулира кроз једну додатну комплексну операцију спој-множи-сабери по итерацији, поред обичне орбите, процена растојања готово да се добија бесплатно уз извршавање теста бекства који би се свакако десио. Стварна уштеда долази од тога колико посла пиксел-по-пиксел тест одсецања омогућава Мариани-Силвер алгоритму да прескочи, а не од тога што је сама процена јефтина посматрана изоловано.

3.6 Теорија пертурбације

Сваки до сада описани кернел директно итерира c у типу `f64` или `f32`, што престаје да функционише чим је увећање толико дубоко да се вредности c суседних пиксела разликују мање него што аритметика двоструке тачности може да разлучи. Та граница се достиже далеко пре режима дубоког увећања који самослична граница из одељка 2.3 чини занимљивим. Теорија пертурбације [13] ово заобилази итерирањем *разлике*, а не апсолутне вредности. Једна референтна орбита високе прецизности Z_n рачуна се једном, за једну референтну тачку близу центра приказа. Сваки други пиксел $c = c_{\text{ref}} + \delta c$ затим се рендерује директним итерирањем много мање величине $\delta_n = z_n - Z_n$.

$$\delta_{n+1} = 2Z_n\delta_n + \delta_n^2 + \delta c, \quad (3.6.1)$$

добијене одузимањем рекурзије саме референтне орбите од рекурзије пиксела. Пошто δ_n остаје мало за већину пиксела близу референтне тачке, оно се може итерирати у обичном `f64` чак и тамо где би за представљање одговарајућег z_n била потребна далеко већа прецизност. Ово значи да се скуп посао високе прецизности из одељка 3.8 плаћа једном по кадру, а не једном по пикселу.



Слика 3.6.1: Стварна референтна орбита у $c_{\text{ref}} = -0.75 + 0.1i$, тачки „долине морских коњица” коју користи и одељак 6.3.2, заједно са две орбите пиксела удаљене за мало δc . Све три орбите остају близу једна другој за приказане итерације, што је управо оно што омогућава да δ_n буде представљено у обичном `f64`, док сама Z_n носи проширену прецизност.

Ово не важи за пикселе чија орбита превише одлута од референце. Чим $|\delta_n|$ нарасте до незанемарљивог дела $|Z_n|$, апроксимација на коју се метод ослања престаје да важи, догађај обично назван *глич* (енг. *glitch*). Рендерер ово открива поређењем $|\delta_n|^2$ са малим фиксним делом $|Z_n|^2$ и означава пиксел чим се та граница пређе. Обична варијанта за сваки означени пиксел једноставно прелази на тачан скаларни кернел из одељ-

ка 3.1, што је увек тачно, али поново плаћа пуну цену по пикселу управо за оне пикселе које је пертурбација требало да избегне директно рачунати. *Ребазирање* (енг. *rebasing*) [7] избегава већину тог прелаза без потребе за другом референтном орбитом. Кад год орбита пиксела поново прође близу координатног почетка, чим $|z_n|^2$ падне испод $|\delta_n|^2$, или кад сачувана референтна орбита истекне, тренутна вредност z_n једноставно се усваја као нова, локална делта од 0, и итерација се од те тачке даље наставља у односу на исту референтну орбиту, поново користећи једну унапред израчунату орбиту као да је ту бесплатно израчуната свежа референца. Рендеровање са *више референци* (енг. *multi-reference*) иде и корак даље. Оно рендерује цео кадар у односу на једну референцу, обележава сваки глич-пиксел, а затим рачуна нову референцу центрирану на први још увек глич-пиксел, пошто се гличеви просторно групишу, суседни пиксели теже да заједно одлутају од дате референце, понављајући овај поступак до 8 референтних орбита пре него што одустане и пређе на тачан кернел за све што остане нерешено.

Директним мерењем, рендеровање пертурбацијом не доноси добитак у пропусности. У испитаном опсегу увећања оно рендерује спорије од директног кернела који замењује, јер тај опсег никада не достиже дубину на којој f64 стварно постаје недовољан, тако да је машинерија пертурбације чист додатни трошак изнад аритметике коју би директни кернел ионако сам могао да обави.

Листинг 3.6.1 показује `perturb_mandelbrot_rebase`, варијанту са ребазирањем. Ажурирање `er/ei` је директан код рекурзије из једначине 3.6.1, а провера `if zn_sq < er * er + ei * ei || m == orbit.len` је сам услов ребазирања: чим квадрат модула тренутне позиције орбите падне испод квадрата модула саме делте, или чим се референтна орбита потроши, тренутна позиција (az, bz) постаје нова делта, а m се враћа на нулу.

```

fn perturb_mandelbrot_rebase(
    orbit: &RefOrbit,
    dc_re: f64, dc_im: f64,
    max_iter: u32,
) -> f32 {
    let mut er = 0.0f64;
    let mut ei = 0.0f64;
    let mut m = 0usize;

    for n in 0..max_iter {
        let zr = orbit.zr[m];
        let zi = orbit.zi[m];

        let two_zr = 2.0 * zr;
        let two_zi = 2.0 * zi;
        let new_er = two_zr * er - two_zi * ei + (er * er - ei * ei) + dc_re;
        let new_ei = two_zr * ei + two_zi * er + (2.0 * er * ei) + dc_im;
        er = new_er;
        ei = new_ei;
        m += 1;

        let az = orbit.zr[m] + er;
        let bz = orbit.zi[m] + ei;
        let zn_sq = az * az + bz * bz;

        if zn_sq > ESCAPE_RADIUS_SQ {
            return smooth_iter(n + 1, zn_sq, max_iter);
        }

        if zn_sq < er * er + ei * ei || m == orbit.len {
            er = az;
            ei = bz;
            m = 0;
        }
    }
    max_iter as f32
}

```

Листинг 3.6.1: Итерација пертурбације са ребазирањем
(src/fractal/perturbation/perturbation_theory.rs).

3.7 Апроксимација степеним редом

Итерација пертурбације из одељка 3.6 и даље извршава по једно ажурирање δ_n по пикселу по итерацији. Апроксимација степеним редом [13, 7] покушава у потпуности да прескочи првих неколико стотина итерација тог посла, за пикселе довољно блиске референтној орбити да је њихово рано понашање предвидиво у затвореном облику.

Уврштавањем претпоставке степеног реда $\delta_n \approx A_n \delta c + B_n \delta c^2 + C_n \delta c^3 + D_n \delta c^4$ у рекурзију пертурбације, једначина 3.6.1, и изједначавањем степена δc добија се рекур-

зија за саме коефицијенте, израчуната једном, заједно са референтном орбитом.

$$A_{n+1} = 2Z_n A_n + 1, \quad B_{n+1} = 2Z_n B_n + A_n^2, \quad (3.7.1)$$

$$C_{n+1} = 2Z_n C_n + 2A_n B_n, \quad D_{n+1} = 2Z_n D_n + (2A_n C_n + B_n^2). \quad (3.7.2)$$

Све док чланови трећег и четвртог реда остају занемарљиви у односу на водећи члан за најгори случај δc у кадру, теме приказа најудаљеније од референтне тачке, ред је ваљана замена за директну итерацију. Рендерер прати последњу итерацију у којој то још важи, за сваки пиксел директно процењује ред у тој итерацији, а затим наставља обичну итерацију пертурбације од те тачке, уместо од нулте итерације.

На сцени за бенчмарк коришћеној изнад за пертурбацију, испоставља се да је прескочиви префикс кратак. Он износи свега неколико десетина итерација од хиљаду, предност од један до три процента, а не велики почетни прескок који апроксимација степеним редом омогућава на већим дубинама.

3.8 Нумеричка прецизност за дубоко увећање

Теорија пертурбације, одељак 3.6, помера проблем прецизности уместо да га уклања. Делте пиксела могу остати у $f64$, али једна референтна орбита у односу на коју се оне рачунају и даље мора да прати апсолутну позицију у комплексној равни, а на екстремном увећању тој позицији је потребно више од приближно 53 бита мантисе које нуди $f64$ да би се разликовала од суседних. Уместо увођења библиотеке за аритметику произвољне прецизности, рендерер представља реалан број проширене прецизности на начин Декерове класичне „double-double” технике [4]: као неевалуирани пар обичних вредности типа $f64$, (hi, lo) , уз инваријанту да је $|lo|$ највише пола јединице последњег места у hi . Стварна вредност је тачан збир $hi + lo$, који се никада заправо не рачуна као један број покретног зареза. Ова „double-double” репрезентација даје приближно 106 бита мантисе, двоструко више него $f64$, док се свака појединачна операција над њом и даље извршава као обична хардверска аритметика покретног зареза.

Два градивна елемента су Кнутов TwoSum [9], кратак низ операција без грањања који рачуна $a + b$ заједно са тачном грешком заокруживања у другом броју, и TwoProduct [17], који исту тачну декомпозицију грешке за множење добија директно из спојене инструкције множења и сабирања. Он рачуна $p = a \times b$, а затим члан грешке као $a \times b - p$ у једној спојеној инструкцији множења и сабирања, без потребе за посредним резултатом двоструке ширине. Сабирање над типом пара сабира горње делове помоћу TwoSum, увија оба доња дела у резултујући члан грешке и поново нормализује другим позивом TwoSum. Множење рачуна тачан производ горњих делова помоћу TwoProduct, додаје унакрсне чланове $hi_a \cdot lo_b$ и $lo_a \cdot hi_b$ као обичне $f64$ производе, који су већ довољно мали да им није потребан сопствени члан грешке, и на исти начин поново нормализује.

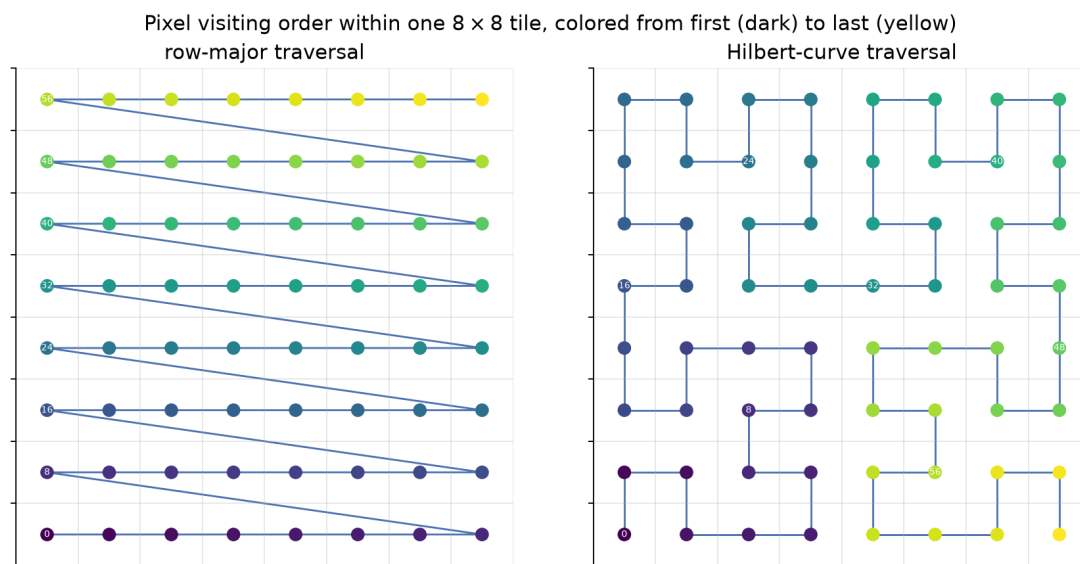
Ова проширена прецизност користи се на тачно једном месту, при рачунању саме референтне орбите, чим тражено увећање пређе фиксни праг од 10^{12} , изнад ког обичан $f64$ више није поуздан за ту орбиту. Свако ажурирање δ_n по пикселу и даље се извршава у обичном $f64$ у односу на ту орбиту, скраћено назад из репрезентације пара приликом чувања, пошто самим делтама никада није ни била потребна додатна прецизност, само дељеној референци. Пошто се референтна орбита рачуна једном по кадру, а не једном по пикселу, приближно двоструко већи аритметички трошак double-

-double операција у односу на обичан f64 остаје занемарљив у односу на укупно време кадра, чак и када је активан.

3.9 Обилазак и локалност

Сваки до сада описан алгоритам одлучује *шта* да се израчуна за пиксел. Овај одељак бави се редоследом којим се пиксели посећују, као и потпуним избегавањем поновног рачунања када претходни кадар већ садржи већи део одговора.

Рендерер дели кадар на квадратне плочице од 64×64 пиксела и расподељује траке плочица по радним нитима процесора. Ова фиксна величина плочице уједно је и јединица посла коју хибридни распоређивач из главе 5 шаље процесору и графичком процесору. Унутар плочице, пиксели се могу посећивати у обичном редоследу по редовима, или дуж Хилбертове криве [8], рекурзивно дефинисане путање која увек прелази на геометријски суседан пиксел и, за разлику од редоследа по редовима, никада на крају реда не скаче назад преко целе ширине плочице. Хилбертов редослед посете за плочицу генерише се једном преплитањем битова и ротирањем квадраната, и поново се користи непромењен за сваку плочицу у кадру, тако да је цена по пикселу само провера у табели, а не поновно рачунање криве.



Слика 3.9.1: Редослед посете пиксела унутар једне плочице, редослед по редовима наспрам Хилбертове криве. Редослед по редовима на крају сваког реда скаче назад преко целе ширине плочице. Хилбертова крива никада не прави корак који није ка геометријски суседном пољу.

Мотивација за испробавање Хилбертовог редоследа уопште јесте локалност кеша. Узастопни пиксели дуж криве такође су просторно блиски на слици, па би, ако цео бафер кадра не стане у кеш, то требало да значи мање избацивања него код обиласка по редовима, који поново посећује околину реда тек након што прође кроз све остале редове између. На хардверу за бенчмарк коришћеном у овом раду, тај очекивани добитак се не јавља. Хилбертов обилазак мерљиво спорије рендерује од обичног редоследа по редовима на свакој испитаној резолуцији, а стопа промашаја кеша последњег нивоа му је виша, а не нижа. Најизгледније објашњење, испитано у глави 7, јесте да бафер кадра на испитаним резолуцијама већ удобно стаје у кеш последњег нивоа процесора,

уз хардверско унапред дохватање (енг. *prefetching*) које успешно ради на приступном обрасцу по редовима, тако да преплитање битова по пикселу код Хилбертове криве постаје чист додатни трошак, без преосталог проблема локалности који би требало да реши. Другачији редослед обиласка, Мортоново или Z-индексирање [16], користи се на другом месту у овом рендереру, али на графичком процесору, а не на овде описаној путањи плочица на процесору. Одељак 4.3 описује како он одржава да нити унутар једног GPU „warp”-а приступају блиској меморији.

Померање приказа (енг. *panning*) уопште не мора да утиче на редослед обиласка, већ само на количину обављеног посла. Када се приказ помери дуж једне осе, већина пиксела претходног кадра поново се јавља на помереној позицији, уместо да их треба поново рачунати. Рендерер копира део старог бафера који се може поново искористити на његову нову позицију једним масовним копирањем меморије и рачуна само новооткривену траку дуж ивице ка којој се приказ померио. Ово претвара поновно рачунање целог кадра у копирање плус рачунање сразмерно растојању померања, не површини кадра, али само за померање дуж једне осе. Истовремено хоризонтално и вертикално померање захтевало би преклапајуће копирање поново искоришћених и новооткривених области, што масовно копирање не омогућава, па се дијагонално померање рачуна директно, пошто делимично поновно коришћење ионако не би уштедело посао.

4. ИМПЛЕМЕНТАЦИЈА НА ГРАФИЧКОМ ПРОЦЕСОРУ

4.1 WebGPU кернели

WebGPU је преносиво од два позадинска решења рендерера за графички процесор. Исти компајлирани шејдер извршава се на сваком графичком процесору са Vulkan, Metal или DirectX драјвером, по цену једног строгог ограничења. WGSL, шејдерски језик у који се WebGPU компајлира, уопште нема тип $f64$, тако да је сваки кернел у овом позадинском решењу искључиво $f32$. За разлику од позадинских решења за процесор и CUDA, ово позадинско решење нема сопствени резервни механизам за дубоко увећање. Остаје унутар истог прага увећања од 10^6 који се кроз цео рад користи за брзу путању $f32$, а све изнад тог прага рендерује се на потпуно другом позадинском решењу.

Унутар тог ограничења, WebGPU кернел блиско прати сопствене $f32$ кернеле процесора. Извршава исте тестове за кардиоиду, период 2 и период 3 из одељка 3.3, уграђене директно у шејдер, и прескаче детекцију периодичности на исти начин као SIMD кернели процесора, из истог разлога: Брентова провера циклуса се не исплати уз додатно гранање на $f32$ гранулацији. Кернел за Манделбровтов скуп дели своју шејдерску датотеку са осталим типовима фрактала у рендереру, које се бирају у реалном времену помоћу целобројног поља у малом униформном баферу параметара приказа и рендеровања, отпремљеном једном по кадру. Две улазне тачке деле овај код кернела. Једна покреће по једну нит по пикселу за цео кадар у редоследу по редовима, у радним групама од 16×16 . Друга уместо тога чита отпремљени низ дескриптора плочица и користи позицију сваке радне групе у распоређивању да изабере једну, проверавајући границе локалне позиције сваке нити у односу на ширину и висину те плочице пре уписивања на њену стварну позицију у баферу кадра. Кроз ову другу улазну тачку шаље своје позиве GPU радник хетерогеног распоређивача (глава 5), пошто му је потребно да у једном позиву рендерује произвољан скуп плочица које бира распоређивач, а не цео кадар.

Посебан шејдер обрађује рендеровање пертурбацијом. Он отпрема референтну орбиту (одељак 3.6) као два $f32$ низа за реални и имагинарни део, заједно са њеним центром, и итерира исту рекурзију делте, једначина 3.6.1, коју користе и кернели на процесору, прелазећи на директну $f32$ евалуацију Манделбровтовог скупа при гличу или када отпремљена орбита истекне. Не врши ребазирање и не покушава поново са другом референцом. За разлику од три варијанте на процесору, WebGPU пертурбација користи само једну референцу. Ребазирање и вишеструке референце додаде би контролни ток који мора идентично да се извршава кроз сваку нит у радној групи да би остао јефтин на графичком процесору, а процењено је да то није вредно за позадинско решење чији је домет ионако ограничен на $f32$.

Враћање резултата хосту кошта више него његово израчунавање. Кернел уписује у складишни бафер на страни графичког процесора. Копирање са уређаја на уређај затим га премешта у други бафер означен за мапирање ка хосту, који се асинхроно мапира и чека, пре него што се његов садржај копира у свеже алоциран бафер на хосту.

При резолуцији 1920×1080 , увећању 1, $\text{max_iter} = 1000$, сам кернел се извршава за 0.29 ms, али цео кадар траје 2.26 ms. Читање назад чини отприлике 90% тога.

4.2 CUDA позадинско решење

CUDA позадинско решење директно циља дискретни NVIDIA графички процесор, уместо да пролази кроз слој преносивости. Кернел се унапред компајлира помоћу nvcc, циљајући архитектуру sm_86, која одговара RTX 3050 генерације Ampere на којој се овај рад бенчмаркује, а резултујући PTX уграђује се у компајлирани извршни фајл, тако да ништа не мора да се компајлира у реалном времену. Две компајлерске опције вреди посебно истаћи. Једна задржава денормализоване бројеве покретног зареза, `--ftz=false`, пошто их формула глатког бојења, одељак 2.4, захтева ради тачности близу прага бекства. Друга у потпуности искључује сажимање спојеног множења и сабирања, `--fmad=false`. Тај други избор је слика у огледалу кернела на процесору из одељка 3.1, који FMA укључује ради брзине и прихвата разлику од једног корака заокруживања у односу на одвојено множење и сабирање. Овде је искључен како би CUDA-ина аритметика заокруживала потпуно исто као и аритметика процесора, жртвујући мало пропусности графичког процесора за слике које су бит-по-бит идентичне између позадинских решења, што је важно када се та два директно пореде (глава 7).

За разлику од WebGPU позадинског решења, ово задржава своје подразумеване кернеле у f64, чиме постиже потпуни паритет са основном варијантом на процесору. Извршава исте тестове за кардиоиду, период 2 и период 3, као и исту детекцију циклуса у Брентовом стилу из одељка 3.1. Испод истог прага увећања од 10^6 , коришћеног свуда другде у овом раду, преузимају посебни f32 кернели, огледајући сопствено распоређивање прецизности процесора. Посебан кернел обрађује пертурбацију, чувајући референтну орбиту и сваку делту по пикселу у f64, за разлику од искључиво f32 путање WebGPU позадинског решења. Он чак ни не прима координате референтне тачке као аргумент. Пошто се орбита увек рачуна за сопствени средишњи пиксел кадра, кернел реконструише c_{ref} директно из геометрије самог кадра. Као и WebGPU позадинско решење, нема ребазирање нити поновни покушај са другом референцом, а ниједно од два позадинска решења не користи double-double аритметику из одељка 3.8. Нивои увећања довољно дубоки да им је она потребна у овој имплементацији и даље се рендерују искључиво на процесору.

Погрешно постављене заставице и погрешна политика прецизности нису хипотетички ризик. Тест периода 3 у f32 кернелу за Манделбровтов скуп некада је позивао f64 верзију функције `in_period3_bulb`, по логици да и сопствени f32 кернел процесора на исти начин позива своју f64 проверу балона, а да се та провера извршава само једном по пикселу, а не једном по итерацији, па не би требало да буде важна. То резонување важи на процесору, где f64 и f32 аритметика коштају приближно исто. На овом графичком процесору не важи. Ampere извршава f64 на отприлике 1/64 пропусности f32, тако да осам f64 операција, извршених једном по пикселу, кошта колико и пропусност издавања инструкција за целу петљу од хиљаду итерација у f32 која их следи. Аблациони тест који је мерио само кернел, без преноса ка хосту, показао је да верзија са f64 провером ради за 0.910 ms, наспрам 0.452 ms након замене правим f32 тестом балона, фактор два, потврђен директним прегледом компајлираног PTX-а. Осам f64 инструкција преживело је у старом кернелу, а нула у новом. Исправка је проверена као бит-по-бит идентична оригиналу, не само визуелно слична, рендеровањем седам приказа изабраних да оптерете управо ону границу коју провера штити, укључујући

приказе центриране директно на луку периода 3 при нивоима увећања до прага прецизности 10^6 , и поређењем сваког од 2,073,600 пиксела. Ниједан се није разликовао, ни у једном приказу. Остварени ефекат на потпуно рендерован кадар, укључујући пренос ка хосту који доминира временом, био је смањење од 15 до 19% на свакој испитаној резолуцији, од 800×600 до 3840×2160 , а иста исправка, будући да је у дељеном коду кернела, бесплатно се применила и на распоређивачево плочичасто слање посла графичком процесору.

Листинг 4.2.1 је позив теста пре исправке, унутар `mandelbrot_f32` кернела: `cr/ci` се проширују на `double` само зато да би позвале `f64` верзију теста. Листинг 4.2.2 је функција која ју је заменила на позивном месту, аритметички идентична осим што остаје у `float` од почетка до краја.

```
if (in_period3_bulb((double)cr, (double)ci)) return (float)max_iter;
```

Листинг 4.2.1: Позив теста периода 3 пре исправке, унутар `mandelbrot_f32`: провера у `double` унутар иначе `float` кернела.

```
__device__ __forceinline__ bool in_period3_bulb_f32(float cr, float ci) {
    float dr = cr - (float)PERIOD3_CENTER_RE;
    float di_pos = ci - (float)PERIOD3_CENTER_IM;
    float di_neg = ci + (float)PERIOD3_CENTER_IM;
    return (dr * dr + di_pos * di_pos < (float)PERIOD3_RADIUS_SQ)
        || (dr * dr + di_neg * di_neg < (float)PERIOD3_RADIUS_SQ);
}
```

Листинг 4.2.2: `in_period3_bulb_f32`, `float` верзија истог теста која је заменила позив изнад (`src/gpu/cuda/fractal.cu`).

Као и код WebGPU позадинског решења, пренос назад ка хосту кошта више него сам кернел. При истој конфигурацији 1920×1080 , кернел троши 0.28 ms, а читање на страни хоста траје 1.33 ms, отприлике 82% укупног кадра од 1.61 ms. Регистровање одредишног бафера на хосту као меморије закључане на страници (енг. *page locked*), преко позива CUDA драјвера `cuMemHostRegister`, омогућава да пренос директно врши DMA у меморију у власништву позиваоца, уместо да прво пролази кроз интерни закачени (енг. *pinned*) бафер драјвера, на шта би га незакачено одредиште иначе приморало. Ако то регистровање буде одбијено, код се враћа на обичну меморију уместо да потпуно откаже. Ово је, на крају, објашњење зашто CUDA-ин кадар од 1.61 ms побеђује WebGPU-ов кадар од 2.36 ms. Када се примени горња исправка, кернели два позадинска решења разликују се за мање од 5%, а готово цела преостала разлика долази од додатног међукопирања тог позадинског решења.

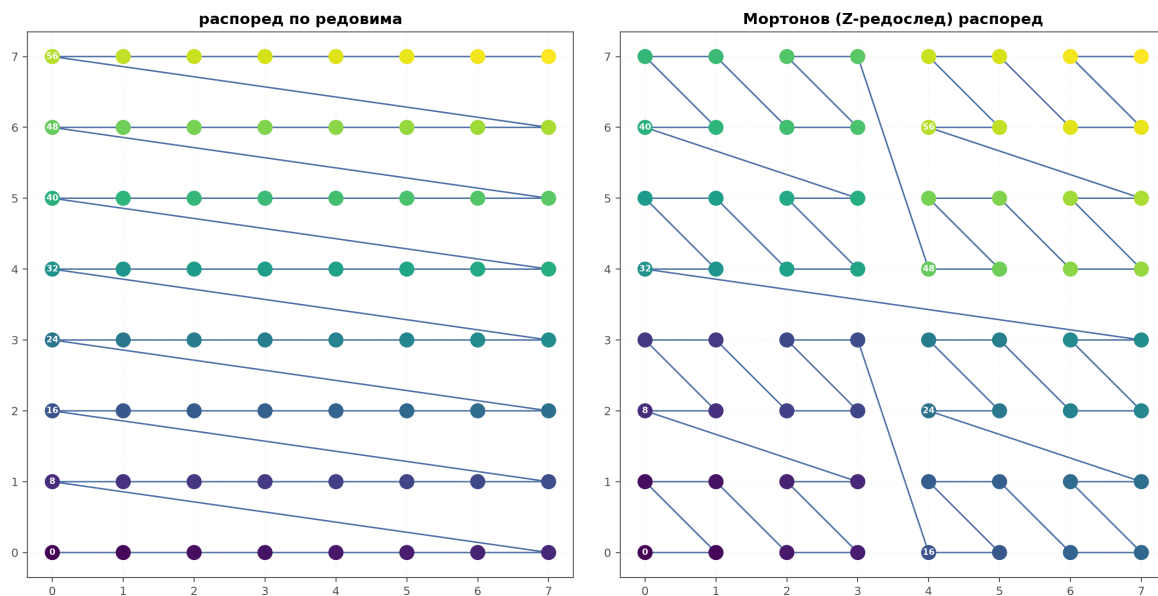
CUDA-ino убрзање у односу на процесор у великој мери зависи од тога у односу на коју бројку процесора се мери. У односу на скаларну `f64` основну варијанту из одељка 3.1, CUDA рендерује сцену за бенчмарк из овог рада отприлике $5.4\times$ брже. У односу на најбољи сопствени кернел процесора, кернел SIMD са два ланца `f32x8` из одељка 3.2, та разлика пада на $2.64\times$, а када се у мерење укључи и корак `colorize()`, извршен на процесору за обе стране, поново пада на $2.09\times$. Кернел графичког процесора и хардвер идентични су у све три бројке. Мења се само страна процесора у поређењу. Та вредност од $2.09\times$ претходи премештању CUDA-иног сопственог корака бојења на уређај (одељак 4.3). Одељак 7.3 поново отвара то поређење са применом те промене. То је управо поента коју је одељак 1.1 извео из литературе [10], сада измерена директно на рендереру из овог рада. Неоптимизована основна варијанта на процесору чини да

графички процесор изгледа неколико пута бржи него што то показује правилно оптимизована варијанта.

4.3 Обилазак на графичком процесору и распоред меморије

CUDA позадинско решење своје кернеле за цео кадар покреће у Мортеновом редоследу, познатом и као Z-редослед, уместо у редоследу по редовима [16]. Равни индекс сваке нити декодира се назад у позицију пиксела (x, y) преплитањем битова два одвојена 16-битна броја у један 32-битни код, тако да индекси који су блиски у покретању одговарају пикселима који су на слици блиски по обе осе истовремено, а не само дуж једног реда. Сама мрежа покретања попуњена је до најмањег обухватног квадрата чија је страница степен двојке око веће димензије кадра, а затим прекривена блоковима нити од 16×16 . Нит чија декодирана позиција падне ван стварног кадра једноставно се враћа без икаквог уписа. Наведено образложење за овакво покретање, уместо покретања по редовима, јесте да је цена алгоритма бекства просторно корелисана (одељак 2.4), па би задржавање просторне блискости нити унутар једног „warp”-а требало да учини њихове бројеве итерација уједначенијим и смањи колико дуго нити „warp”-а које брже заврше чекају најспорију.

Редослед покретања нити преко плочице 8×8 , обојен од првог (тамно) до последњег (жуто) нити



Слика 4.3.1: Редослед покретања нити преко плочице 8×8 . Редослед по редовима на крају сваког реда скаче назад преко целе ширине плочице, тако да узастопни индекси нити на тим границама завршавају далеко једни од других по оси y . Мортонев редослед много чешће задржава узастопне индексе унутар истог квадранта 2×2 или 4×4 , својство локалности за које одељак 4.3 тврди да би требало да помогне да бројеви итерација унутар једног „warp”-а остану уједначени.

Директним мерењем, ово образложење се не исплати на овом хардверу. Упарено поређење у истом процесу са обичном мрежом по редовима, помоћу исте математике кернела, само покренуте кроз распоређивачеву плочичасту улазну тачку, показује да Мортонско покретање издаје нешто више од двоструко више нити него што има пиксела, 4,194,304 наспрам 2,073,600 за кадар 1920×1080 , пошто попуњавање до квадрата

отприлике удвостручи мрежу. Медијана трошка кроз седам поновљених извршавања износила је око +1,9%, при чему су два од седам извршавања заправо ишла у прилог Мортоненом покретању. То је неразлучиво од шума. Вишак нити завршава у једној непрекидној области иза стварне ивице кадра, тако да се цели блокови одмах повлаче, без икаквог узалудног посла по пикселу, због чега удвостручавање броја покренутих нити у сваком случају готово ништа не кошта. Попуњавање је готово бесплатно, али, према овом мерењу, то важи и за редослед који је требало да омогући. Ово је исти закључак до ког је одељак 3.9 дошао тестирајући Хилбертову криву на процесору, редослед обиласка који испуњава простор, изабран из разлога локалности, директно упоређен са обичном алтернативом, за који се показало да не помера бројку на хардверу који је стварно мерен.

Хетерогени распоређивач уопште не користи Мортоневу путању за плочице које шаље графичком процесору. Уместо тога, покреће их кроз плочица-кERNEL, по један блок по плочици, изабран индексом z блока, при чему се за сваку нит проверавају границе у односу на ширину и висину њене плочице. Друга варијанта тог плочица-кERNELа додатно носи одредишни померај по плочици, за упис у један густо запакован излазни бафер, уместо на сопствену позицију сваке плочице у целом кадру, коришћен када распоређивач графичком процесору додели расути подскуп слике, а не непрекидну област слике (глава 5).

Испод оба редоследа обиласка, оба позадинска решења уписују у исту врсту бафера: равни низ по редовима, са по једним $f32$ по пикселу, који садржи сирову, могуће разломљену вредност бекства из једначине 2.4.1, а не боју, уз 4 бајта по пикселу. То износи 8.29 MB за кадар 1920×1080 мерен кроз ову главу. Претварање тога у приказану слику, хистограм, кумулативну расподелу и читавање из палете, посебан је пролаз преко истог бафера, извршен као сопствени пар кERNELа на CUDA страни, огледајући двостепену поделу рендер-па-обоји коју већ користи цевовод на процесору.

И одељак 4.1 и одељак 4.2 измерили су читање на страни хоста на 80 до 90% укупног времена кадра, за читав ред величине више него што су избори редоследа обиласка у овом одељку помакли ту бројку. Хибридни распоређивач из главе 5 изграђен је управо око тог трошка читања.

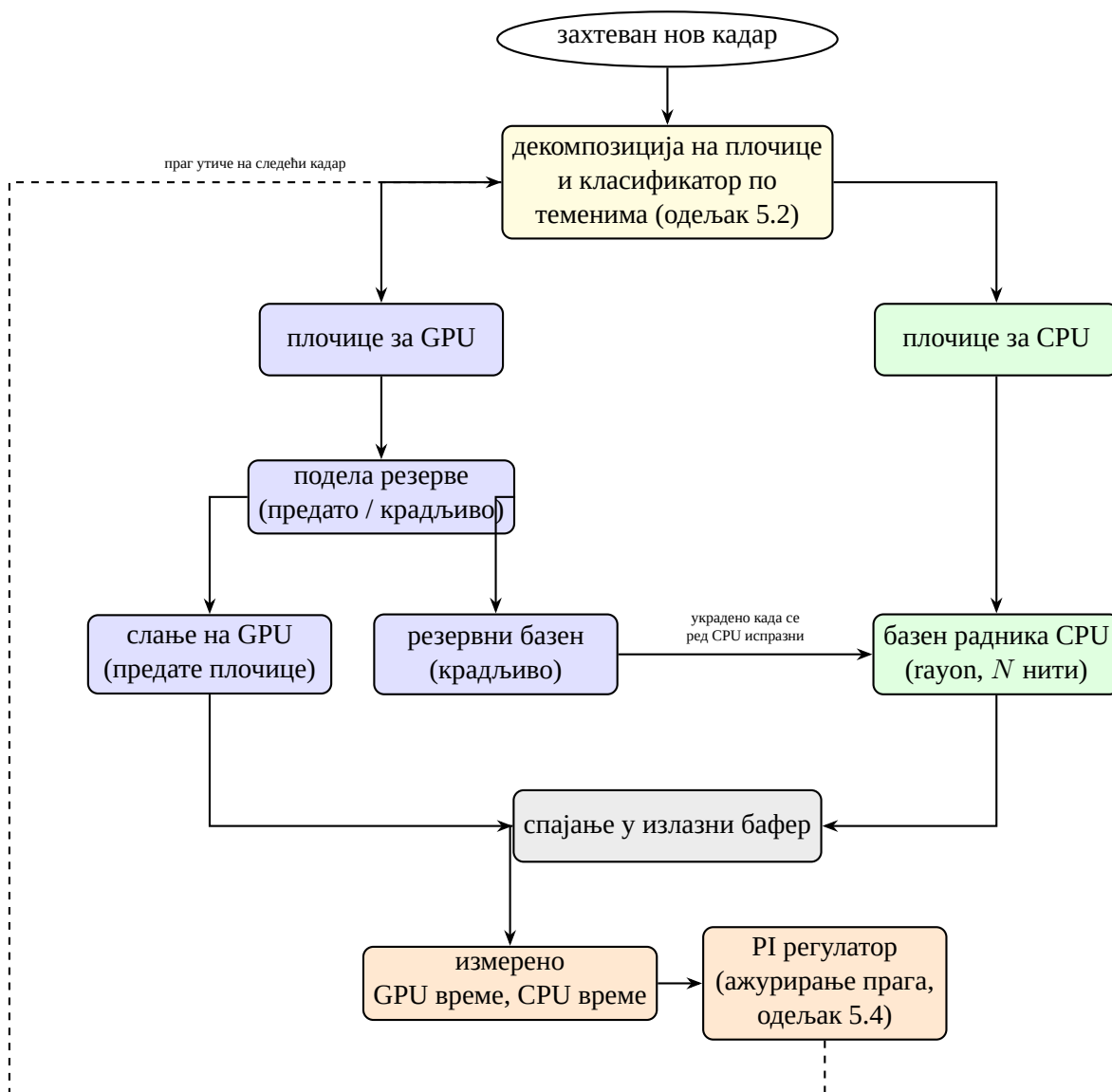
5. ХИБРИДНИ РАСПОРЕЂИВАЧ ЗА ПРОЦЕСОР И ГРАФИЧКИ ПРОЦЕСОР

5.1 Мотивација и дизајн

Глава 4 завршила се једном бројком. Читање на страни хоста чини 80 до 90% кадра на графичком процесору, код оба позадинска решења, и ниједна оптимизација на нивоу кернела не дотиче тај удео. Ништа у алгоритмима рендеровања из овог рада не смањује тај трошак, јер он нема никакве везе са алгоритмом. То је једноставно цена преноса готовог бафера преко PCIе магистрале. Оно што алгоритам по пикселу *може* да промени јесте који уређај уопште рачуна пикселе тог бафера, а два уређаја нису подједнако добра за сваки пиксел. Нити графичког процесора извршавају се у „warp”-овима од 32, у лок-кораку (енг. *lockstep*), па ако једна нит у „warp”-у достигне граничну итерацију док друга побегне после десет итерација, цео „warp” наставља да ради док се не заврши његов најспорији члан. То је исти трошак маскираних трака који SIMD кернели из одељка 3.2 плаћају при ширини од осам, осим што заглављени „warp” на графичком процесору не може усред лета да преузме други посао онако како то може слободно језгро процесора. Језгро процесора нема такво ограничење. Свака његова нит независно обрађује своје пикселе, тако да хаотична област са високим бројем итерација то језгро кошта време сразмерно само сопственим пикселима, и ништа више. Области слике близу границе скупа, где се бројеви бекства суседних пиксела нагло разилазе, стога су јефтине на процесору, а упоредно скупе на графичком процесору. Униформне области, дубоко у скупу или далеко изван њега, јефтине су на оба уређаја, пошто лок-корак извршавање графичког процесора тамо нема преостале казне за дивергенцију коју би платило. Распоређивач у овој глави управо ту асиметрију искоришћава. Он сваки део слике усмерава ка ономе уређају који мање кошта за *тај* део, у духу адаптивног хетерогеног мапирања између процесора и графичког процесора [11], уместо да цео кадар рендерује на једном уређају или га дели по фиксном правилу које занемарује садржај.

Дизајн има четири покретна дела. Прво, пре него што се рендерује иједан пиксел, кадар се распарчава на плочице и свака плочица се јефтино класификује као погодна за графички процесор или за процесор, узорковањем само њена четири темена (одељак 5.2). Друго, та класификација се претвара у стваран посао. Плочице класификоване за графички процесор шаљу се њему, док скуп радника на процесору истовремено празни плочице класификоване за процесор из дељеног реда, при чему оба уређаја раде истовремено кроз цео кадар, уместо да један чека на другог (одељак 5.3). Треће, подела одлучена пре него што иједан уређај рендерује иједан пиксел не може у реалном времену да зна оптерећење ниједног од њих, па се део плочица графичког процесора задржава као посао који свако слободно језгро процесора може да „украде” чим му се сопствени ред испразни, чиме се исправља привремено спор графички процесор унутар истог кадра (одељак 5.3). Четврто, исправка само унутар кадра није довољна ако је један уређај *доследно* бржи или спорији него што претпоставља статички праг класификатора, пошто интерактивно рендеровање изнова рендерује кадар при сваком померању и увећању. Стога измерено GPU и CPU време сваког кадра храни PI регулатор

који поново подешава праг класификатора за наредни кадар, затварајући петљу између кадрова, а не унутар једног (одељак 5.4).



Слика 5.1.1: Ток података хибридног распоређивача по кадру. Жута боја означава класификатор, плава путању графичког процесора, зелена путању процесора, сива корак спајања, а наранџаста повратну петљу између кадрова која се затвара кроз PI регулатор. Испрекидана стрелица једина је грана која прелази границу кадра; свака пуна стрелица се дешава унутар кадра који се тренутно рендерује.

5.2 Декомпозиција плочица и класификација

Задатак класификатора је да за област кадра чији ниједан пиксел још није рендерован одлучи да ли та област припада графичком процесору или процесору, и то без трошења ни приближно онолико времена колико би заправо коштало њено рендеровање. Ради тако што узоркује темена. Кернел алгоритма бекства се процењује само на четири темена плочице, а степен у ком се те четири вредности разилазе служи као замена за то колико дивергентни ће вероватно бити пиксели унутар ње. Кадар је прво прекривен мрежом ћелија једнаке величине у редоследу по редовима (128×128 пиксела

подразумевано), класификованих независно и паралелно, пошто класификација једне ћелије не дотиче ниједно дељено стање и не зависи од исхода ниједне друге ћелије.

Унутар ћелије, четири вредности темена дају нормализовани распон,

$$s = \frac{\max(i_1, i_2, i_3, i_4) - \min(i_1, i_2, i_3, i_4)}{N}, \quad (5.2.1)$$

где су $i_1, \dots, i_4$ бројеви итерација бекства четири темена, а N је граница итерација, што чини вредност упоредивом између различитих граница итерација, пошто представља део буџета итерација, а не сирови број итерација. Ако s падне испод прага, плочица се класификује за графички процесор, пошто се њена темена довољно добро слажу да се пиксели између њих претпостављају униформним. То је иста претпоставка о локалности коју Мариани-Силвер тест ивице (одељак 3.4) прави о унутрашњости правоугаоника, овде примењена да усмери посао између уређаја, уместо да га потпуно прескочи. Ако је s једнако прагу или изнад њега, плочица се дели на два дела дуж своје дуже осе, иста подела по дужој оси коју користи и Мариани-Силвер тест ивице, на две плочице од којих се свака рекурзивно класификује, поново користећи два узорка темена на дељеној ивици од родитеља и процењујући само два заиста нова темена настала поделом, уместо поновног узорковања сва четири за сваку половину. Рекурзија се зауставља на минималној величини плочице (16×16 пиксела подразумевано). Плочица те величине класификује се за процесор без обзира на свој распон, пошто даља подела престаје да се исплати без обзира колико дивергентна плочица и даље изгледа. Сам праг није фиксан. Одељак 5.4 објашњава одакле долази.

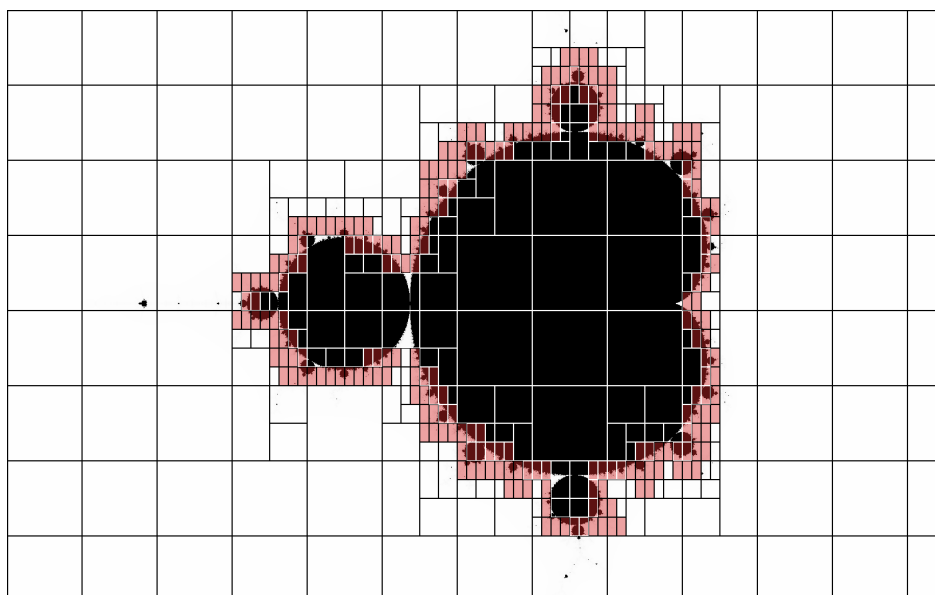
Листинг 5.2.1 су функције иза једначине 5.2.1. `sample_corners` је једини пиксел-по-пиксел рад који класификатор уопште обавља, тачно четири позива кернела по ћелији или подплочици.

```
fn corner_spread(corners: [f32; 4], max_iter: u32) -> f32 {
    let mut max_val = f32::MIN;
    let mut min_val = f32::MAX;
    for &v in &corners {
        max_val = max_val.max(v);
        min_val = min_val.min(v);
    }
    (max_val - min_val) / (max_iter.max(1) as f32)
}

fn sample_corners(ctx: &PartitionCtx, x0: u32, y0: u32, tw: u32, th: u32) ->
    [f32; 4] {
    let re0 = ctx.pg.re_start + x0 as f64 * ctx.pg.re_step;
    let re1 = ctx.pg.re_start + (x0 + tw - 1) as f64 * ctx.pg.re_step;
    let im0 = ctx.pg.im_start + y0 as f64 * ctx.pg.im_step;
    let im1 = ctx.pg.im_start + (y0 + th - 1) as f64 * ctx.pg.im_step;
    [
        pixel(ctx.fractal, re0, im0, ctx.julia_c, ctx.max_iter),
        pixel(ctx.fractal, re1, im0, ctx.julia_c, ctx.max_iter),
        pixel(ctx.fractal, re0, im1, ctx.julia_c, ctx.max_iter),
        pixel(ctx.fractal, re1, im1, ctx.julia_c, ctx.max_iter),
    ]
}
```

Листинг 5.2.1: Узорковање темена и нормализовани распон
(src/scheduler/classifier.rs).

Излаз класификатора су две одвојене листе правоугаоника плочица, једна намењена графичком процесору, а друга процесору, из којих одељак 5.3 директно шаље посао. Директним мерењем, цео овај пролаз кошта 0.03 до 0.32 ms у опсегу од плитког до умереног увећања, занемарљиво у односу на времена кадрова која му следе, а која трају по неколико милсекунди. Узорковање само четири темена по плочици, уместо сваког пиксела, чини да трошак одлучивања где рендеровати остане мали у односу на трошак стварног рендеровања. Подела коју ово производи такође прати садржај сцене, уместо да примењује неки фиксни однос. На сцени за бенчмарк из овог рада, удео пиксела на процесору расте са отприлике 3% при увећању 1 на отприлике 14% при увећању 100, како граница скупа, једина област довољно дивергентна да не прође тест темена, почиње да испуњава већи део кадра.



Слика 5.2.1: Подела на плочице класификатора приказана на рендеру целог кадра Манделбрововог скупа. Плочице класификоване за графички процесор остављене су необојене. Плочице класификоване за процесор освенчене су црвено.

Слика 5.2.1 чини ту концентрацију видљивом. Плочице процесора формирају танку траку која прати сам облик границе, а не чврсту масу, а рекурзија иде најдубље управо тамо где тај облик најмање личи на праву линију. Шпицеви на местима где се главна кардиоида спаја са пратећим балонима издвајају се као мале групе плочица минималне величине унутар шире траке. То је исти ефекат који стоји иза удела процесора од 3% до 14% наведеног изнад. Колико год сеfino граница набира на размерама испод једног пиксела, мрежа плочица је у стању да је разложи само до резолуције минималне плочице, тако да удео процесора остаје везан за то колико од видљивог кадра граница пресеца, а не за то колико је та крива заправо сложена.

5.3 Расподела посла и синхронизација

Листа графичког процесора коју производи класификатор не предаје се графичком процесору у целости. Завршни део те листе, подразумевано 20%, уместо тога се издваја у посебан резервни базен, а остатак, предате плочице, је оно што се заиста одмах шаље графичком процесору. И резервни базен и листа за процесор, сада сопствени

ред посла, представљају дељено стање о које се боре нити, из којег свака радна нит процесора може да узима.

По једна радна нит извршава се по доступном језгру процесора, а свака радна нит ради у петљи. Прво преузима плочицу из свог сопственог реда за процесор, и тек кад се тај ред испразни посеже у резервни базен, тако да се резервисани посао графичког процесора дотиче тек након што се потроши сопствени, додељени удео радне нити. Док те радне нити раде истовремено, графички процесор се покреће на предатим плочицама у једном пакету, тако да ниједан уређај не чека другог за већи део кадра. Када се то покретање заврши, распоређивач проверава колико је плочица још увек не-преузето у резервном базену. Ако их остане бар 64, радници на процесору очигледно до њих још нису дошли, још увек заузети сопственим редом, па се исплати још једно покретање графичког процесора да заврши остатак, уместо да се чека да их процесор на крају испразни. У супротном, остатак је довољно мали да сопствени трошак другог покретања графичког процесора не би вредео, па се једноставно оставља радницима на процесору да га настављају да краду. Читање с графичког процесора које следи копира цео бафер кадра без обзира на то који је удео графички процесор заиста рендеровао, пошто би смањивање тог копирања захтевало да удео графичког процесора буде један непрекидан опсег редова, а не расути скуп плочица.

Када и сваки радник на процесору и покретање графичког процесора заврше, рендероване плочице сваког уређаја копирају се на своју стварну позицију у дељени излазни бафер. Током самог рендеровања није потребно ниједно закључавање по пикселу или по плочици, само кратак тренутак у ком радник преузима плочицу из реда. Плочице које рендерују графички процесор и радник на процесору увек су дисјунктне, тако да корак спајања који следи нема ниједан сукоб да разрешава.

5.4 Адаптивно балансирање оптерећења помоћу PI регулатора

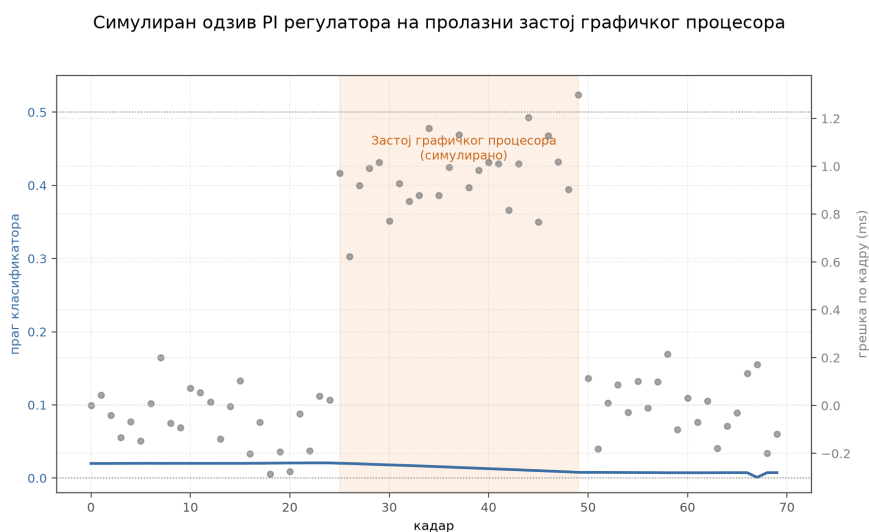
Праг класификатора (одељак 5.2) замена је за то колико дивергентна ће вероватно бити унутрашњост плочице, а не директно мерење тога који уређај ту плочицу рендерује брже, а права вредност те замене зависи од садржаја сцене и од релативног оптерећења два уређаја на начине које ништа пре самог рендеровања не може унапред знати. Регулатор повратне спреге уместо тога затвара петљу споља. Он посматра стварно измерено GPU и CPU време сваког кадра и благо помера праг тако да оба уређаја остану уравнотежена у наредном кадру.

Регулатор прати једну означену грешку по кадру, $\text{error} = t_{\text{gpu}} - t_{\text{cpu}}$, разлику између измереног времена рендеровања на графичком процесору и на процесору за тај кадар, и ажурира праг као

$$\text{threshold} \leftarrow K_P \cdot \text{error} + K_I \cdot \overline{\text{error}}, \quad (5.4.1)$$

уз $K_P = 0.0005$ и $K_I = 0.00005$, а затим се резултат ограничава на опсег $[0.001, 0.5]$. Позитивна грешка значи да је графички процесор тог кадра био спорији од процесора, а једначина 5.4.1 на то одговара снижавањем прага. Пошто је сам тест класификатора $\text{spread} < \text{threshold}$, нижи праг је строжи захтев за униформношћу, па мање плочица пролази тест, а више их се усмерава ка процесору у наредном кадру, растеређујући уређај који је заостајао. Негативна грешка помера праг у супротном смеру. Само пропорционални члан реагује искључиво на тренутни кадар и оставио би устаљену пристрасност

некоригованом ако је један уређај *доследно* бржи, а не само шумно бржи од кадра до кадра. Интегрални члан $\overline{e_{TOT}}$, покретни просек преко последњих 16 кадра, а не неограничена текућа сума, исправља ту преосталу пристрасност, а истовремено дозвољава да утицај једног шумног кадра избледи после шеснаест ажурирања, уместо да траје заувек. Пропорционално појачање десет је пута веће од интегралног, тако да регулатор најбрже реагује на кадар који је управо измерен, а на интегрални члан се ослања само за оно што пропорционални сам не исправи. Ограничење на $[0.001, 0.5]$ спречава да велика, устаљена неравнотежа, попут застоја драјвера графичког процесора, одвуче праг у крајност у којој се класификатор своди на усмеравање свега ка једном уређају без обзира на садржај, чиме би се одбацила управо она свест о садржају коју је одељак 5.2 и изграђен да пружи.



Слика 5.4.1: Симулација једначине 5.4.1 кроз 70 синтетичких кадра, са привременим застојем графичког процесора убаченим између кадра 25 и 50. Праг пада док застој траје, усмеравајући већи део кадра ка процесору, а прозор интеграла од шеснаест кадра наставља да га вуче надоле још неколико кадра након што застој престане, пре него што се пропорционални члан поново стабилизује на шуму средње вредности нула. Ово извршавање случајно достиже доњу границу од 0.001, илуструјући управо случај за који је горње ограничење и постављено.

Листинг 5.4.1 је цео регулатор, свега двадесетак линија. Ажурирање `self.integral` рачуна покретни просек преко `history`, кружног бафера последњих 16 грешака, а последње две линије тела `update` су директно једначина 5.4.1, укључујући ограничење опсега.

```

pub struct ThresholdController {
    pub threshold: f32,
    integral: f32,
    history: [f32; 16],
    head: usize,
}

impl ThresholdController {
    pub fn update(&mut self, gpu_ms: f32, cpu_ms: f32) {
        let error = gpu_ms - cpu_ms;
        self.history[self.head] = error;
        self.head = (self.head + 1) % self.history.len();
        self.integral = self.history.iter().sum::<f32>() / self.history.len()
            as f32;

        const K_P: f32 = 0.0005;
        const K_I: f32 = 0.00005;
        self.threshold -= K_P * error + K_I * self.integral;
        self.threshold = self.threshold.clamp(0.001, 0.5);
    }
}

```

Листинг 5.4.1: PI регулатор прага класификатора
(src/scheduler/controller.rs).

Пошто интерактивно рендеровање изнова рендерује кадар при сваком померању и увећању, ова повратна спрега ради непрекидно, а не као једнократни корак калибрације. Ажурирање се извршава на крају сваког хетерогеног рендеровања, а праг који оставља за собом тачно је оно што класификатор из одељка 5.2 чита на почетку самог следећег. Заједно са крајом унутар кадра из одељка 5.3, распоређивач исправља неравнотежу оптерећења на две временске скале истовремено. Крађа апсорбује графички процесор који заостаје унутар једног кадра, а регулатор апсорбује графички процесор који заостаје кроз кадрове, преобликујући саму класификацију. Глава 7 мери колико од трошка читања с којим је ова глава почела та комбинација заиста надокнади, и где су два уређаја довољно блиска по брзини да исправка уопште буде важна.

6. ЕКСПЕРИМЕНТАЛНА МЕТОДОЛОГИЈА

Глава 7 извештава о времену кадра, пропусности и понашању при скалирању за свако позадинско решење изграђено у главама 3, 4 и 5. Ова глава дефинише услове под којима су те бројке добијене. Одељак 6.1 обрађује софтверски стек и начин на који је конфигурисан и мерен. Одељак 6.2 обрађује машину. Одељак 6.3 обрађује коришћене фрактале, резолуције и приказе, а одељак 6.4 обрађује извештаване величине.

6.1 Технолошки стек

6.1.1 Rust

Rust је компајлирани системски језик без сакупљача отпадака (енг. *garbage collector*). Безбедност меморије и одсуство трка над подацима спроводе се у време компајлирања, кроз проверу власништва и позајмљивања, а не путем сакупљача који ради у реалном времену. Та гаранција је важна за рендерер чија је извештавана јединица време кадра у милисекундама. Имплементација са сакупљачем отпадака захтевала би да се свако мерење раздвоји на паузе сакупљања и алгоритами трошак пре него што иједна бројка у глави 7 нешто значи, а овај рендерер нема такво раздвајање да прави. Компајлер своди код на нативни машински код кроз LLVM, тако да се генерички и итераторски заснован Rust код компајлира у исти машински код као и еквивалентна ручно написана петља. Блокови `unsafe`, коришћени само за позиве CUDA драјвера, једини су излаз из провера компајлера.

Пројекат циља издање 2024 и гради се кроз две конфигурације у време компајлирања, примењене уједначено на сваку бројку у овом раду.

- `release` профил, `opt-level = 3`, `lto = "thin"`, „танка” оптимизација у време линковања кроз читав граф зависности, и `codegen-units = 1`, једна јединица генерисања кода, чиме се време компајлирања жртвује за потпуну видљивост оптимизатора у програм. Ниједно мерење у овом раду не потиче из `debug` верзије, која је за читав ред величине спорија и није употребљива основа за поређење, и
- `-C target-cpu=native`, постављено преко `rustflags`, што компајлеру дозвољава да циља сваки скуп инструкција који процесор референтне машине (одељак 6.2) заиста поседује, укључујући AVX2 инструкције у које се компајлирају SIMD кернели `f32x8/f64x4`. Конзервативан подразумевани циљ уопште не би генерисао те инструкције.

Четири крејта носе имплементацију на страни процесора. Овде се наводи за шта се сваки од њих користи, а не како, пошто одељак 3.1 и одељак 3.2 већ обрађују саме кернеле.

- `rayon`, базен нити са крађом посла (енг. *work stealing*) иза сваког кернела на процесору, скаларног и SIMD подједнако,
- `wide`, преносиви SIMD векторски типови `f64x4/f32x8`,
- `num-complex`, аритметика комплексних бројева коју итерирају кернели алгоритма бекства, и

- `image`, за упис рендерованих кадрова у PNG ради провере излаза.

6.1.2 GPU технологије

Две технологије графичког процесора стоје иза два GPU кернела упоређена у овом раду, а разликују се по томе шта овом раду дозвољавају да контролише у време компајлирања. WebGPU је преносив API за израчунавање и графику, стандардизован независно од било ког појединачног произвођача. Драјвер га пресликава на било који нативни API који платформа изложи, на референтној машини Vulkan. Његови шејдери су написани у WGSL-у и компајлирани у реалном времену од стране драјвера, тако да један изворни код кернела ради непромењен на сваком уређају способном за WebGPU, по цени изостанка контроле у време компајлирања над машинским кодом који на крају производи сопствени компајлер драјвера. Овај рад приступа WebGPU-у преко `wgpu` (верзија 22), Rust имплементације коју такође користе Firefox и Deno.

CUDA је затворена платформа компаније NVIDIA за израчунавање. Кернели се унапред компајлирају помоћу `nvcc` у PTX, NVIDIA-ину стабилну посредну репрезентацију, уграђену у компајлирани извршни фајл, тако да ништа не мора да се компајлира у реалном времену. Пет заставица конфигурише то компајлирање, примењених идентично на сваки CUDA кернел у овом раду.

- `--gpu-architecture=sm_86`, директно циљајући генерацију Ampere референтног графичког процесора (одељак 6.2), уместо генеричког циља који би драјвер морао да преводи при учитавању,
- `-O3`, највиши ниво оптимизације алата `nvcc`,
- `--ftz=false`, чиме се задржавају денормализовани бројеви покретног зареза, потребни за тачност формуле глатког бојења близу прага бекства,
- `--prec-div=true`, дељење пуне прецизности, и
- `--fmad=false`, чиме се искључује сажимање спојеног множења и сабирања, тако да CUDA-ина аритметика заокружује потпуно исто као и кернели на процесору, уз измерени трошак испод 1% пропусности кернела када се уклоне остали трошкови прецизности (одељак 4.2).

Овај рад приступа CUDA 12 преко `cudarc`, Rust везивног слоја (енг. *binding*) за API CUDA драјвера.

6.1.3 Технологија бенчмаркинга

Три алата мере рендерер, сваки прилагођен другачијој намени.

- `criterion`, статистички крејт за бенчмаркинг, извршава главни пакет (`fractal_bench`). Свака група добија двосекундно загревање, које се одбацује, а затим пет секунди мерења, и извештава средњу вредност са интервалом поверења од 95%, а не једним узорком. Поновљена извршавања номинално идентичног кернела разликовала су се за више од 50% унутар једног позива алата `criterion`.
- `bench_runner`, наменски израђен извршни фајл који детерминистички рендерује једну фиксну конфигурацију, за упаривање са `perf stat` или за издавање машински читљивог JSON-а, и

- скуп самосталних извршних фајлова-сонди (`gpu_probe`, `ptx_variants`, `sched_probe`, `sched_overhead`, `sched_deepzoom`, `tiled_dispatch_probe`, `adapters`), од којих сваки изолује једну фазу или једну хипотезу коју једна бројка времена кадра сама по себи не може раздвојити: на пример време кернела од времена читања на страни хоста, ефекат једне компајлерске заставице од ефекта друге, или трошак распоређивача од квалитета његових одлука.

`perf stat` обезбеђује бројаче кеша и грана на процесору, NVIDIA Nsight Compute обезбеђује бројаче заузетости и ефикасности „warp”-ова на графичком процесору, а RAPL (енг. *Running Average Power Limit*) бројачи потрошње енергије пакета, заједно са потрошњом снаге коју извештава `nvidia-smi`, обезбеђују бројке о енергији у одељку 6.4.4.

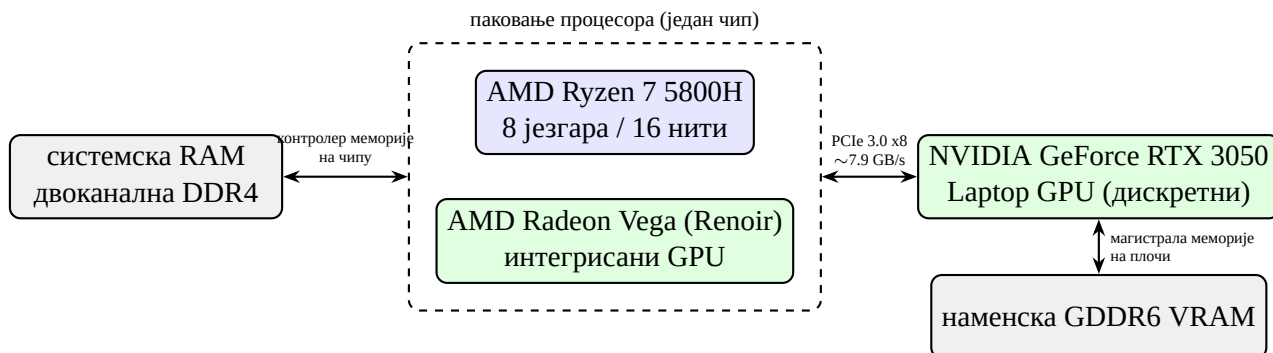
Четири постојећа рендерера прегледана у одељку 2.2 не деле ништа од овог алата. Сваки је за потребе овог рада опремљен еквивалентним двослојним оквиром за мерење, циљем `criterion_bench` заснованим на Google Benchmark-у и CLI позивом `bench_runner` који директно позива сопствени компајлирани кернел тог пројекта, тако да свих пет рендерера ради из једне командне линије и производи упоредив JSON. Сопствени модел рада са нитима сваког рендерера мери се онакав какав је изграђен, а не нормализован на базен алата `rayon`. Поређење циља алгоритамску пропусност по стварно израчунатом пикселу, па се свака пречица коју пројекат користи да избегне директно израчунавање пиксела, попут Хаос-овог праћења границе (енг. *boundary tracing*) или FractalNow-ове интерполације по четворкама (енг. *quad interpolation*), искључује. Обе замењују израчунавање интерполацијом на делу слике, што би мерило колико добро пројекат нагађа, а не колико брзо ради његов кернел.

6.2 Хардвер

Сва мерења у овом раду, кроз рад називана референтном машином, потичу са једног лаптопа, изграђеног око следећег.

- процесора AMD Ryzen 7 5800H, осам језгара Zen 3, шеснаест нити преко двоструког SMT-а,
- графичког процесора NVIDIA GeForce RTX 3050 Laptop GPU, Ampere (`sm_86`), CUDA 12.0, драјвер 580.173.02, PCIe 3.0 x8 (теоријски 8 GB/s), и
- интегрисаног графичког процесора AMD Radeon Vega, кодног имена Renoir.

Слика 6.2.1 приказује како су ове компоненте заправо повезане. Чип Ryzen носи интегрисани графички процесор Radeon Vega на истом паковању, тако да оба деле исту двоканалну DDR4 системску RAM меморију преко једног контролера меморије на чипу, потпуно без PCIe преноса између њих. RTX 3050 је посебан, дискретан део, доступан само преко PCIe 3.0 x8, са сопственом наменском GDDR6 VRAM меморијом коју процесор не може директно да адресира, управо она веза чији трошак одељак 4.3 и глава 5 у великој мери настоје да заобиђу у овом раду.



Слика 6.2.1: Топологија компоненти референтне машине. Интегрисани графички процесор дели паковање процесора и системску RAM меморију без икаквог трошка преноса. Дискретни графички процесор доступан је само преко PCIe магистрале, са сопственом одвојеном VRAM меморијом.

Избор адаптера алата `wgrr` тихо се враћа на интегрисани графички процесор када није инсталиран NVIDIA Vulkan драјвер, чиме потцењује трошак читања с графичког процесора, пошто интегрисани део дели меморију са процесором и уопште не плаћа PCIe пренос. Свака сесија бенчмарка бележи изабрани адаптер при покретању, што се проверава пре него што се његове бројке употребе.

Референтна машина је лаптоп, а не наменска машина за бенчмарк, и термички је промењива. Идентични бенчмаркови процесора или графичког процесора, покренути у одвојеним сесијама без икакве промене кода између њих, помакли су се за 15 до 30% само услед термичког стања. Свака размера и бројка убрзања у глави 7 рачуна се унутар једног извршавања. Апсолутне бројке у милисекундама ипак носе ту варијансу.

6.3 Бенчмаркови и тест сцене

6.3.1 Фрактали, резолуције и бројеви итерација

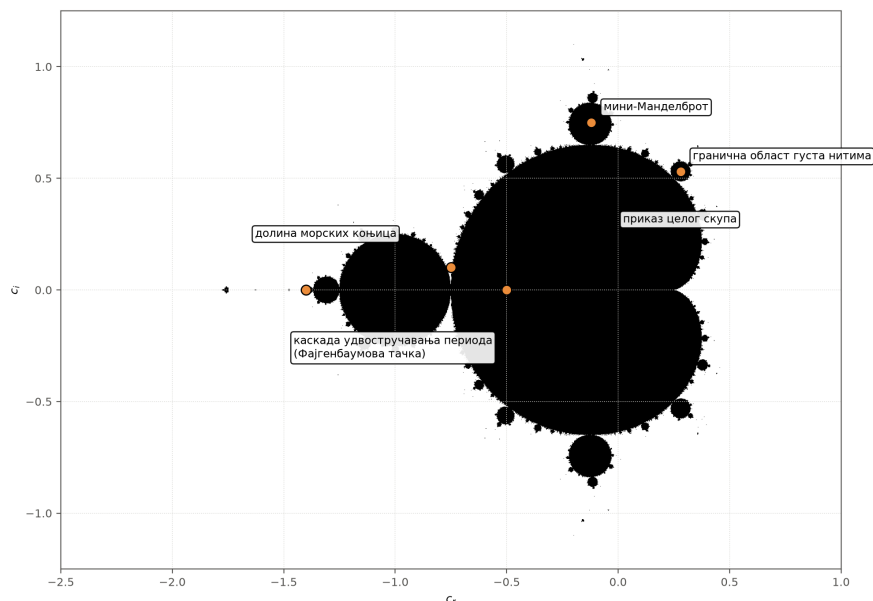
Манделбровов скуп је једини циљ бенчмарка у овом раду. Рендерер такође имплементира Жулијине, Њутнове и Нова фрактале на истој машинерији алгорита бекства, али је свака оптимизација у главама од 3 до 5, тестови кардиоиде и балона, процена растојања, Мариани-Силвер подела, као и рад на SIMD и GPU кернелима, развијена и мерена конкретно на Манделбрововом скупу, и управо о томе извештава остатак овог рада.

Пакет бенчмаркова тестира Манделбровов скуп на три резолуције, 800×600 , 1920×1080 и 3840×2160 , изабране да потврде да добит технике важи независно од величине кадра. Осим ако сама серија мерења не каже другачије, `max_iter` је фиксирано на 1000. Један изузетак је провера конвергенције у односу на површину Манделбрововог скупа, у литератури процењену на приближно 1.50659177, која варира сам `max_iter`, од 100 до 20000.

6.3.2 Прикази

Већина бенчмаркова рендерује цео видљиви скуп из његовог природног центра ($c = -0.5$, увећање 1), мешајући унутрашње, граничне и брзо-бежеће пикселе приближно у пропорцијама какве показује интерактивна сесија. Тамо где трошак технике конкретно зависи од тога колико кадра чини граница, а не унутрашњост, пакет уместо тога користи пет фиксних узорачких тачака које обухватају главну кардиоиду, највећи

балон, мини-Манделброт и област густу нитима, како резултат не би могао бити артефакт једног погодног приказа са мало границе у њему.



Слика 6.3.1: Пет фиксних узорачких тачака, изабраних да обухвате приказ целог скупа, граничну област густу нитима, мини-Манделброт и каскаду удвостручавања периода ка Фајгенбаумовој тачки дуж негативне реалне осе. Тачка „долине морских коњица” истовремено служи и као референтни центар за серије увећања у одељку 6.3.3.

6.3.3 Серије увећања

Три серије мерења испитују понашање зависно од увећања. Плитка серија (увећање $1, 10^2, 10^4$) остаје испод прага 10^6 на ком и кернели за процесор и они за графички процесор прелазе са f32 на f64. Дубока серија (увећање од 10^4 до 10^{15}) прелази тај праг, на фиксној референтној тачки „долине морских коњица” ($c = -0.75 + 0.1i$), изабраној зато што њена граница остаје довољно густа, на сваком нивоу увећања у серији, да класификатору плочица пружи стваран посао. Трећа серија (увећање од 1 до 10^{12} , иста референтна тачка) вреднује теорију пертурбације у односу на обичан f64.

6.3.4 Поређење између рендерера

Поређење са четири постојећа рендерера ограничено је на оно што свих пет пројеката дели, кернел за Манделбровтов скуп, при усклађеној резолуцији, броју итерација и приказу, мерено као сирово време итерације по пикселу, а не као потпуно обојени кадрови, пошто се рендерери разликују по томе колико посла мапирања боје је стопљено у тај корак.

6.4 Метрике

6.4.1 Време кадра и пропусност

Време кадра извештава се као средња вредност узорка алата `criterion`, са његовим интервалом поверења. Пропусност, број рендерованих пиксела по милисекунди, добија се из истог мерења преко механизма бројања елемената алата `criterion`, а не изводи накнадно. Равна крива пропусности кроз резолуције представља радну дефини-

цију „сасвим паралелног” (енг. *embarrassingly parallel*) кернела у овом раду. Крива која се савија указује на трошак који расте са величином кадра, а не само са бројем пиксела.

6.4.2 Ефикасност израчунавања

Поређења између кернела додатно се исказују у GFLOPS-има, по једној фиксној конвенцији. Свака итерација канонског ажурирања $z = z^2 + c$ рачуна се као осам операција покретног зареза, без урачунавања сажимања спојеног множења и сабирања. Конвенција је поједностављење, а не дословно бројање инструкција, али управо њена уједначена примена, и на сопствене кернеле овог рендерера и на сва четири конкурентска пројекта, чини бројку у GFLOPS-има упоредивом између пет независно написаних кодних база.

6.4.3 Поређење контролисано по прецизности

Свако извештено убрзање наводи прецизност, f32 или f64, при којој су обе стране радиле. Упаривање брзог f32 кернела на графичком процесору са спором f64 основном варијантом на процесору вештачки увећава привидни добитак, независно од стварне пропусности било ког уређаја, ефекат који Ли и др. документују за поређења процесора и графичког процесора уопште [10]. Сваки однос графичког процесора и процесора у овом раду наводи се најмање двапут: једном у односу на наивну скаларну основну варијанту, а једном у односу на најбољи доступан сопствени кернел процесора при подударачујућој прецизности.

6.4.4 Паралелно скалирање и енергија

Скалирање по нитима извештава се као убрзање и паралелна ефикасност у односу на сопствено једнонитно извршавање истог рендерера, никада између различитих рендерера, пошто се модели нитовања структурно разликују између пројекта. Енергетска ефикасност извештава се одвојено од брзине, као гигафлопси по вату, из RAPL бројача потрошње енергије пакета (за процесор) и потрошње снаге коју извештава `nvidia-smi` (за графички процесор) током дужег извршавања, пошто спорији уређај није нужно и мање ефикасан.

6.4.5 Тачност и микроархитектонске метрике

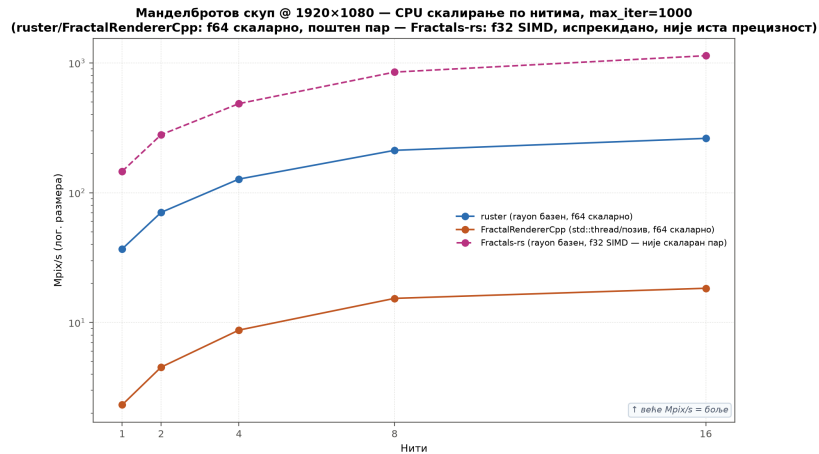
Две додатне метрике примењују се тамо где питање није о брзини. Где оптимизација треба да мења само перформансе, а не излаз, тачност се проверава бројањем разлика по пикселу између варијанти кернела; нула разлика кроз цео кадар је услов пре објаве бројке брзине. Где хипотеза тиче понашања на графичком процесору, на пример дивергенције „warp”-ова у граничним областима, то разрешавају заузетост и ефикасност „warp”-ова из `Nsight Compute`, независно од утицаја на време кадра.

7. РЕЗУЛТАТИ И ДИСКУСИЈА

Ова глава представља мерења која поставка из главе 6 производи. Свака слика потиче са ту дефинисане референтне машине, а сваки овде наведени однос прати правила поређења контролисаног по прецизности и унутар истог извршавања, изложена у одељку 6.4. Већина слика у овој глави генерисана је директно алатом `aggregate_bench.py` овог пројекта, описаним у одељку 6.1, из једног архивираног извршавања бенчмарка, а не поново нацртана ручно, тако да бројка која изгледа мало другачије од оне већ наведене у ранијој глави представља термичку варијансу између извршавања, врсту коју описује одељак 6.2, а не противречност. Глава прати редослед којим су главе од 3 до 5 увеле основне технике: прво основну варијанту на процесору и SIMD, затим два позадинска решења за графички процесор, цео цевовод од почетка до краја, хибридни распоређивач, теорију пертурбације и на крају резултате поређења између пројекта и нумеричке валидације. Два резултата носе више тежине од осталих. Слика 7.2.1 у одељку 7.2 јединствено је мерење у које се улива свака ранија глава оптимизације, пропусност рендеровања за свако позадинско решење изграђено у овом раду, једно поред другог, а одељак 7.4 је место где се хибридни распоређивач, централни допринос овог рада, проверава наспрам алтернативе простог избора бржег уређаја и наспрам поделе коју би теоријски требало да може да достигне.

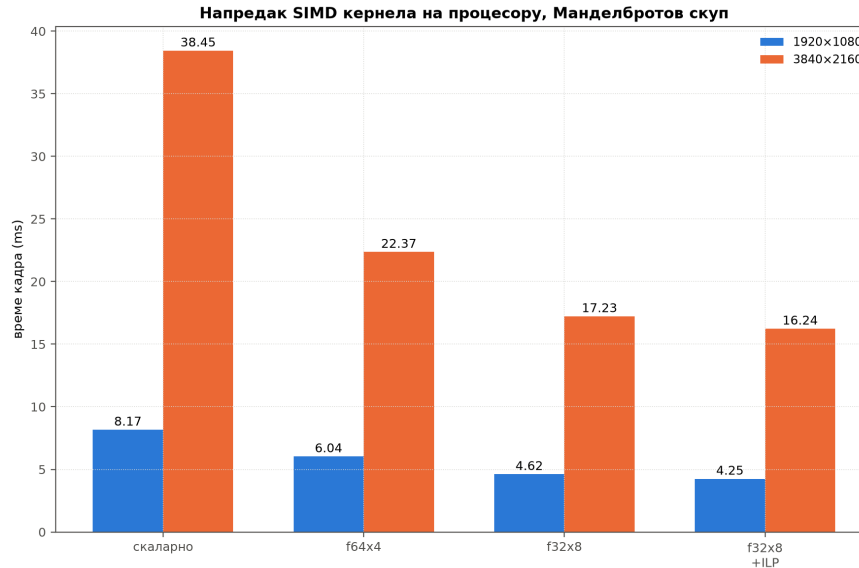
7.1 Основна варијанта на процесору и SIMD скалирање

Слика 7.1.1 потврђује скалирање по нитима које описује одељак 3.1. `ruster`-ово паралелно блоковање редова помоћу `rayon`-а остаје близу праве линије на графику логаритамске размере кроз осам нити, и тек код шеснаест се савија, потпис симултаног вишенитног извршавања (SMT), где друга логичка нит на већ заузетом физичком језгру купује само део пуног језгра, а не још једно удвостручавање. `FractalRendererC++` скалира лошије при сваком броју нити, делом зато што покреће и придружује свеж `std::thread` по кадру, уместо да поново користи базен, трошак имплементације наведен у одељку 6.3.4, који ово поређење укључује у бројку по нити, уместо да га издваја.



Слика 7.1.1: Скалирање по нитима на процесору, ruster наспрам FractalRendererCpp, Манделбров скуп, 1080р, скаларни f64 на обе стране.

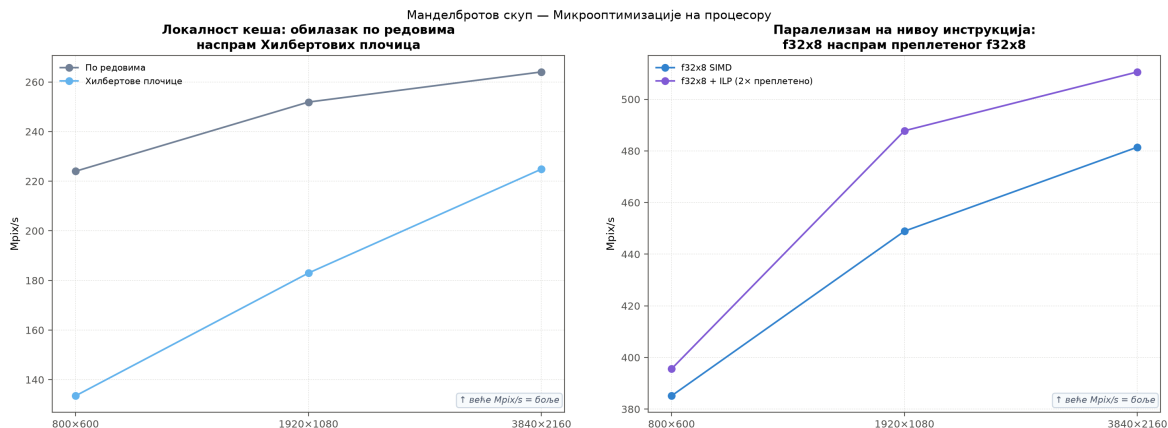
Слика 7.1.2 приказује напредак SIMD кернела из одељка 3.2, мерен на 1080р и 4К. Сваки корак, f64×4, f32×8, а затим кернел са два ланца f32×8+ILP, снижава време кадра, а образац се готово идентично понавља на обе резолуције, доказ да је добитак својство кернела, а не одређене величине кадра. Убрзање од скаларне варијанте до f32×8+ILP износи 1.92× на 1080р и 2.37× на 4К, при чему корак паралелизма на нивоу инструкција доприноси мање од десетине тога. Одељак 3.2 мали остатак приписује томе што тестови кардиоиде и балона већ уклањају већину пиксела који би иначе попунили цевовод другог ланца.



Слика 7.1.2: Напредак SIMD кернела на процесору, Манделбров скуп, 1080р и 4К.

Слика 7.1.3 проверава још два избора на страни процесора наспрам обичне основне варијанте са блоковањем редова у rayon-у. Леви панел је питање редоследа обиласка из одељка 3.9, Хилбертов обилазак наспрам обичног по редовима, а измерена разлика директно потврђује тврдњу тог одељка. Хилбертов обилазак рендерује спорије на свакој испитаној резолуцији, отприлике 224 наспрам 133 Mpix/s при 800 × 600, а разлика се сужава, али се не затвара до 3840 × 2160, без икаквог компензујућег добитка локалности кеша за додатни трошак преплитања битова по пикселу. Десни панел је питање

паралелизма на нивоу инструкција из одељка 3.2 посматрано само за себе, преплетени кернел $f32 \times 8 + ILP$ наспрам обичног $f32 \times 8$, и преплетени кернел је бољи на свакој испитаној резолуцији, од уске разлике од 3% при 800×600 до отприлике 9% на 1080p, у складу са добитком који захтева довољно независног посла по пикселу у лету да би се исплатило књиговодство другог ланца.



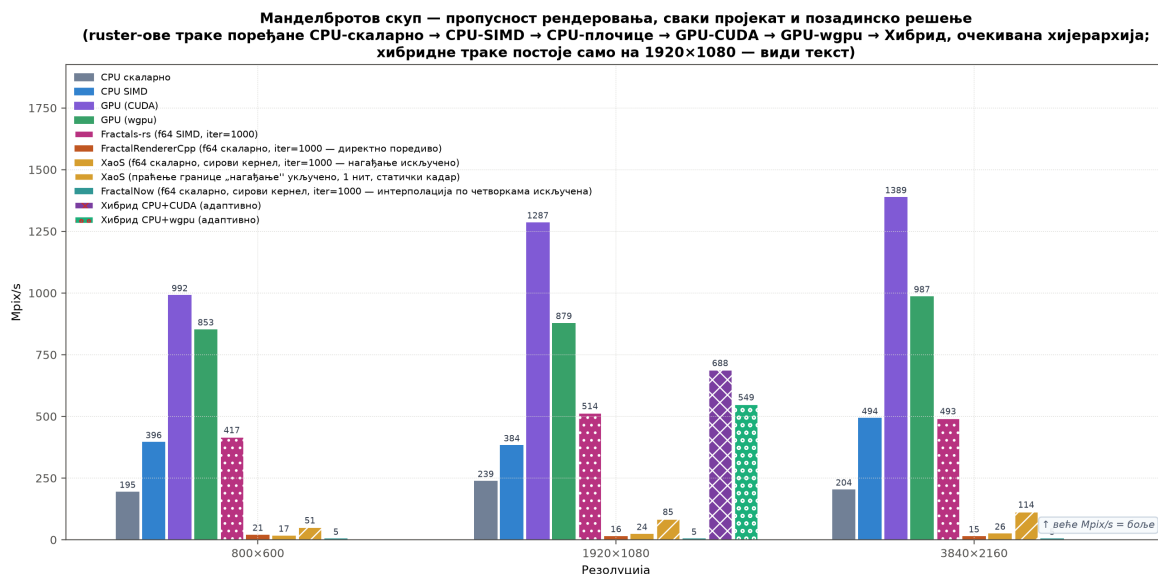
Слика 7.1.3: Микрооптимизације на процесору, Хилбертов наспрам обилазка по редовима и обичан наспрам преплетеног $f32 \times 8$, Манделбровов скуп.

7.2 Позадинска решења за графички процесор

Слика 7.2.1, прва у овом одељку, средишња је тачка ове главе. Свака измерена оптимизација постоји да би померила једну бројку: колико пиксела позадинско решење рендерује у секунди. Она ставља свако позадинско решење изграђено у овом раду на исте осе: скаларни CPU, SIMD CPU, CUDA, wgpu, и сва четири конкурентска пројекта из одељка 6.3.4, Fractals-rs, FractalRendererCpp, XaoS и FractalNow, на три резолуције, тако да је цео напредак од процесора до графичког процесора видљив на једном месту, уместо расутог по читавој глави одвојених панела. XaoS и FractalNow мерени су на свом сировом кернелу по пикселу, са принудно искљученим праћењем границе и интерполацијом по четворкама, како одељак 6.3.4 објашњава, а не на мерењу стварног интерактивног мотора било ког пројекта. Друга, шрафирана трака за XaoS поново укључује праћење границе: 50.8, 85.3 и 114.4 Mpix/s кроз три резолуције, $21 \times$ до $36 \times$ у односу на сопствени сирови кернел, и већ испред сопствене вишенитне траке сировог кернела, упркос томе што ради на једној нити, пошто је путања која ради са нитима у `btrace.cpp` искључена узводно. То је један статички кадар без претходног кадра из кога би се нагађало, тако да мери само алгоритам праћења границе и попуне (енг. *flood fill*), а не додатно поновно коришћење између кадрова које доноси стварни аниматор увећања у XaoS-у, и није упоредив ни са једном другом, вишенитном траком на овој слици.

CUDA предњачи над сваким другим позадинским решењем на свакој резолуцији, wgpu је други, а разлика између два GPU позадинска решења остаје приближно сразмерна величини кадра, у складу са фиксним трошком кернела по пикселу, а не фиксним трошком по кадру који би доминирао било којим од њих. Врхунска пропусност, при 3840×2160 , достиже отприлике 1389 Mpix/s на CUDA-и. Две црвене траке хибридног распоређивача постоје само на 1080p, пошто та група бенчмаркова варира

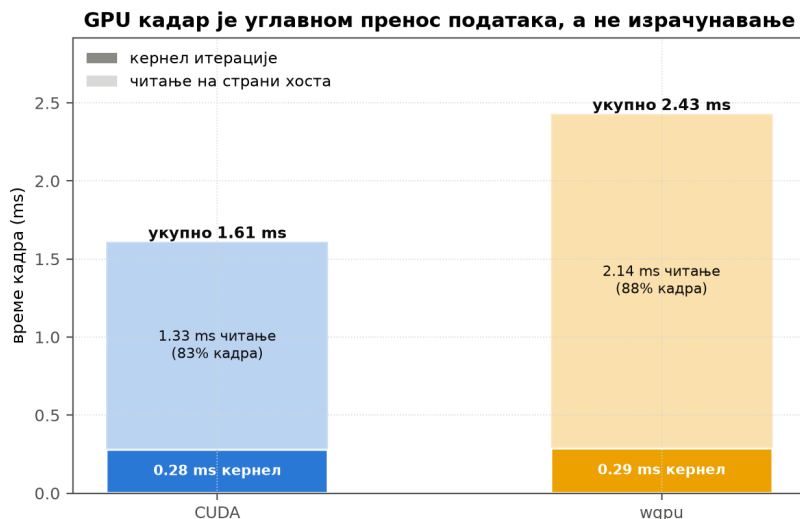
увећање при једној фиксној резолуцији, а не резолуцију при једном фиксном увећању, а одељак 7.4 их чита директно, а не преко ове слике.



Слика 7.2.1: Пропусност рендеровања, свако позадинско решење ruster-a и сва четири конкурентска пројекта, Fractals-rs, FractalRendererCpp, Xaos, FractalNow, три резолуције, Манделбров скуп, увећање 1. Кључно мерење ове главе. Свака CPU и GPU оптимизација од главе 3 до главе 4 овде се читава као растојање између једне и следеће траке. Шрафирана трака за Xaos представља његов стваран алгоритам „нагађања” праћењем границе, једна нит, један статички кадар, неупоредив са вишенитним тракама око ње.

Читане слева удесно унутар било које групе резолуција, траке прате цео пут оптимизације овог рада. Од скаларног до SIMD-а је рад главе 3, од SIMD-а до било ког GPU позадинског решења је рад главе 4. Предност CUDA-е над SIMD-ом на процесору, отприлике $2.5\times$ до $3.4\times$ у зависности од резолуције, повраћај је улагања у изградњу GPU позадинског решења уопште, а остатак овог одељка објашњава где унутар те CUDA бројке заправо одлази време.

Слика 7.2.2 издваја зашто CUDA побеђује у кадру. Два кернела разликују се за мање од 5%, али читање на страни хоста чини 83% CUDA кадра и 88% wgpr кадра, а wgpr на то троши отприлике 0.8 ms више, у складу са додатним међукопирањем са уређаја на уређај које CUDA-ин једини позив `dtoh_sync_sopu` избегава, како одељак 4.2 објашњава.

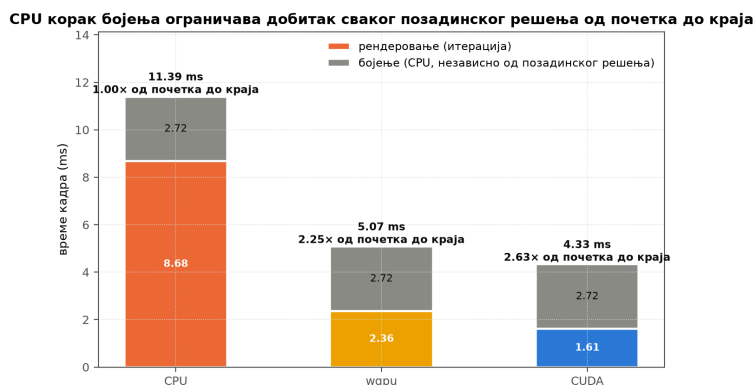


Слика 7.2.2: Расподела времена кадра на графичком процесору, кERNEL наспрам читања на страни хоста, CUDA наспрам wgpu, 1920×1080 , увећање 1.

Другим речима, оба GPU позадинска решења већи део кадра троше на враћање пиксела назад ка хосту, а не на њихово рачунање. То преформулише питање одељка 7.3 пре него што је уопште постављено. Ако је GPU позадинско решење већ доминирано преносом података, а не итерацијом, додавање корака `colorize()` на страни процесора преко тога може ту неравнотежу само учинити видљивијом, не мањом, осим ако и тај корак не пређе на графички процесор, што је, за CUDA, и учињено, како описује одељак 4.3.

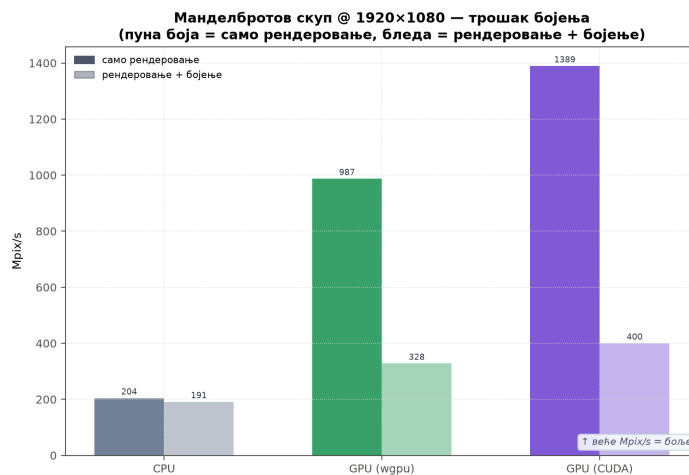
7.3 Цевовод од почетка до краја

Слика 7.3.1 додаје корак `colorize()`, мерен пре него што је било које GPU позадинско решење могло да га прескочи, извршен на процесору подједнако за свако позадинско решење. Предност графичког процесора над процесором нагло се сужава када се укључи тај фиксни корак од 2.72 ms, са отприлике $5.4\times$ само на кораку рендеровања на $2.09\times$ за CUDA и $1.72\times$ за wgpu, од почетка до краја. Од CUDA цевовода од 5.19 ms, отприлике 0.4 ms отпада на итерацију фрактала, а преосталих 4.8 ms на пренос података и бојење. Управо је то оно што је премештање `colorize()` на графички процесор учинило једином оптимизацијом са највећим утицајем на коју је ово мерење указало, већом од исправке fp64 балона из одељка 4.2 и већом од свега што распоређивач из главе 5 може да надокнади. CUDA-ин `colorize()` у међувремену је премештен на уређај кроз `colorize_kernel` и `render_and_colorize`, описане у одељку 4.3, тачан до пиксела у односу на верзију за процесор и 54% до 63% бржи од почетка до краја при плитким и средњим увећањима, тамо где је бојење раније доминирало временом кадра. wgpu и хетерогени распоређивач и даље боје на процесору. Распоређивач то ради нужно, пошто његов спојени бафер постоји само на страни хоста, како објашњава глава 5, тако да горе описани добитак остаје отворен за та два пута.



Слика 7.3.1: Време ценова од почетка до краја, рендеровање плус бојење, 1920×1080 , увећање 1. Мерено са бојењем извршеним на процесору за свако позадинско решење, пре него што је CUDA-ин сопствени корак бојења премештен на уређај, види текст.

Слика 7.3.2 раставља исти корак по позадинском решењу, само рендеровање наспрам рендеровања плус бојење, из истог мерења у ком је свако позадинско решење бојило на процесору. Фиксни трошак `colorize()` однео је сразмерно већи залогај код GPU позадинског решења него код процесора, пошто је сваки пут реч о истом кораку на страни процесора, отприлике петину сопствене пропусности процесора, али ближе две трећине било ког GPU позадинског решења, исти Амдалов ефекат који слика 7.3.1 већ показује исказан као апсолутно време, а не као пропорција. Управо је то ефекат који је премештање бојења на графички процесор учинио вредним за CUDA-у.



Слика 7.3.2: Трошак бојења по позадинском решењу, само рендеровање наспрам рендеровања плус бојење, Манделбров скуп.

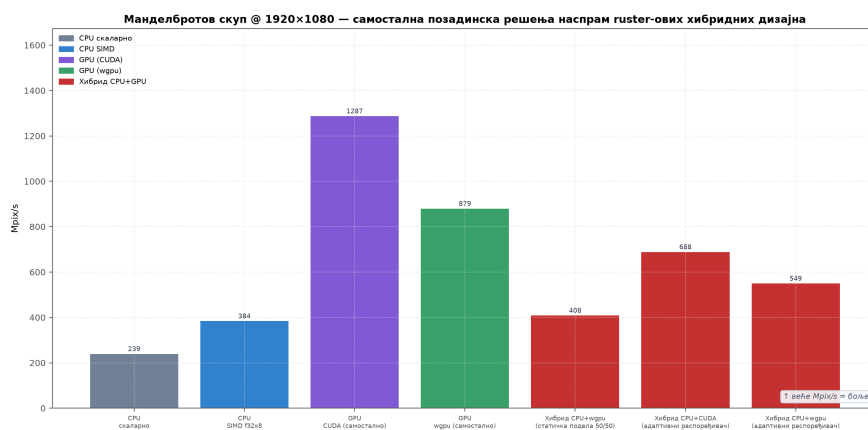
Распоређивач из главе 5 постоји управо да би повратио оно време које је овај одељак управо приказао кроз две слике, па се овом главом даље бави управо то.

7.4 Хибридни CPU+GPU распоређивач

Овде се централна тврдња овог рада проверава наспрам података. Глава 5 изградила је распоређивач који у реалном времену прати оба уређаја и усмерава плочице према измереном трошку, уместо да слепо дели кадар. Две слике овде су доказ да ли се

та машинерија исплати: на плитком увећању, где би требало да се мучи, и на дубокој серији увећања, где је осмишљена да побеђује.

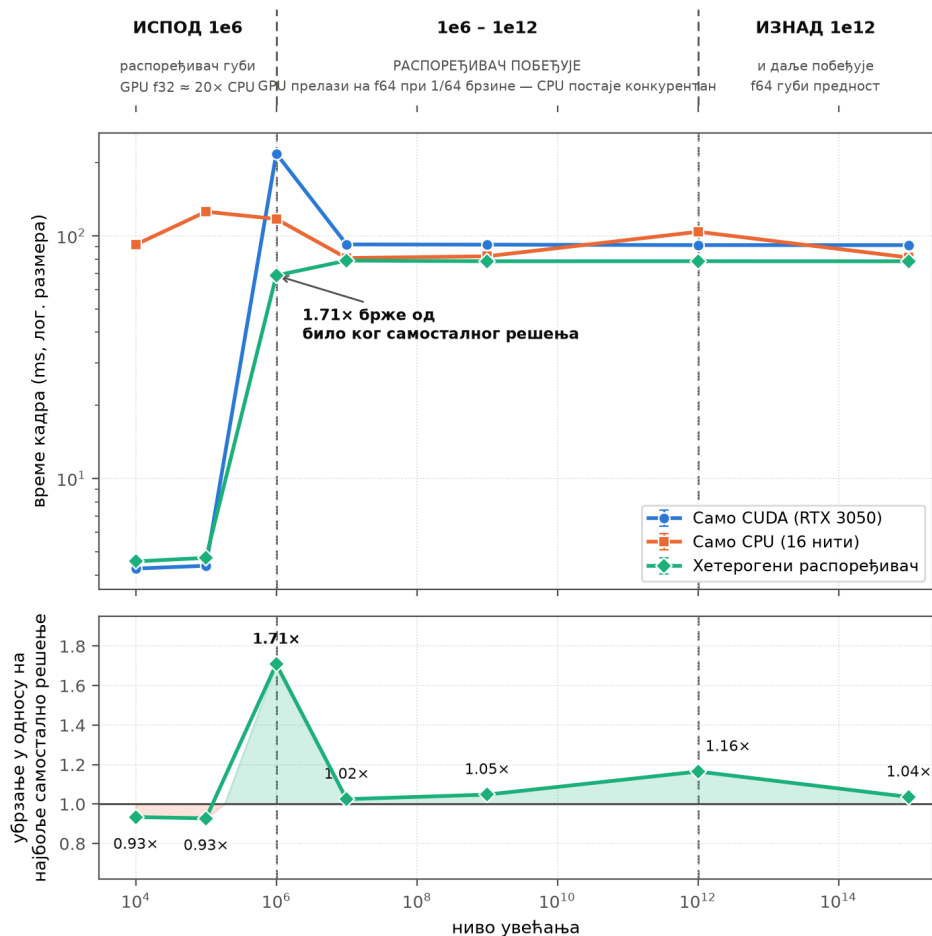
Слика 7.4.1 мери распоређивач из главе 5 наспрам оба самостална позадинска решења и наивне статичке поделе, при увећању 1. Статичка подела 50/50 процесор плус *wgri*, без икакве класификације, најгора је активна конфигурација на графику, 408 наспрам *wgri*-овог сопственог самосталног 879 *Mpix/s*. То је управо неуспех којим одељак 5.3 мотивише класификатор узорковањем темена: равна подела оставља бржи уређај да чека спорији. Адаптивни распоређивач надокнади већи део тог губитка, 688 *Mpix/s* на CUDA-и и 549 на *wgri*-у, али и даље заостаје за коришћењем било ког графичког процесора самог. Свака конфигурација распоређивача губи од простог коришћења бољег графичког процесора самог при овом увећању, аритметика из одељка 5.1 директно се показује: сопствена машинерија кошта фиксан износ, а време кернела које она може да смањи мали је део кадра који је већ доминираан графичким процесором. Укратко, при плитком увећању нема довољно неравнотеже између уређаја да би је распоређивач могао да исправи.



Слика 7.4.1: Самостална позадинска решења наспрам статичке поделе и адаптивног распоређивача, Манделбров скуп 1080p, увећање 1.

Дубоко увећање је другачији режим, и управо ту заправо лежи централни резултат овог рада. Слика 7.4.2 проширује исто поређење са увећања 10^4 до 10^{15} . Тачно на прагу прецизности 10^6 око ког одељак 6.3.3 варира, CUDA-ин сопствени прелазак са *f32* на *f64* накратко је поставља изнад процесора, 218 наспрам 130 ms, пре него што се оба уређаја од 10^7 надаље устале унутар отприлике $1.1 \times$ једно од другог. Управо ту, на том укрштању, распоређивач се исплати. При увећању 10^6 он побеђује не само боље самостално позадинско решење, процесор при том конкретном увећању, него и идеалну пропорционалну поделу, 74.3 наспрам 81.6 ms, добитак од $1.75 \times$ над бољим уређајем, што је директан доказ централне тврдње овог рада: класификација узорковањем темена усмерава скупе плочице ка процесору, уместо да слепо дели кадар, а то усмеравање вреди више од равне поделе, чак и када је равна подела теоријски доступан плафон. Добици после 10^7 мањи су, отприлике $1.13 \times$ до $1.15 \times$, и остављају отприлике 43% доступног простора неискоришћеним, по сопственим бројкама балансера оптерећења, пошто идеална подела при тим увећањима достиже отприлике 45 ms, док хибридни распоређивач и даље мери близу 79 ms, проблем подешавања у режиму који овај пакет бенчмаркова тек сада покрива, а не структурни проблем.

Адаптивни распоређивач побеђује управо тамо где GPU губи своју брзу f32 путању



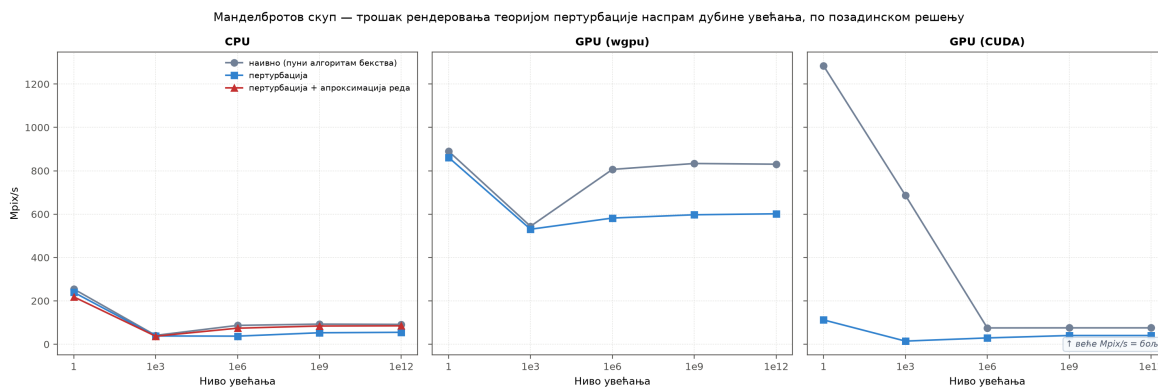
Слика 7.4.2: Хибридни распоређивач наспрам самосталних позадинских решења и идеалне пропорционалне поделе, кроз дубину увећања, Манделбровов скуп, „долина морских коњица”. Добитак од $1.75 \times$ при 10^6 најјаснији је доказ овог рада да управо класификација узорковањем темена, а не само постојање два уређаја, представља допринос хибридног распоређивача.

Две слике заједно одговарају на питање којим је глава 5 почела. Распоређивач је вредан своје сложености само тамо где су уређаји довољно неуравнотежени да усмеравање нешто значи. Губи при плитком увећању јер нема ништа око чега би усмеравао, а побеђује управо тамо где је одељак 5.1 предвидео да ће, на литици прецизности, где један уређај изненада кошта много више по плочици од другог.

7.5 Теорија пертурбације

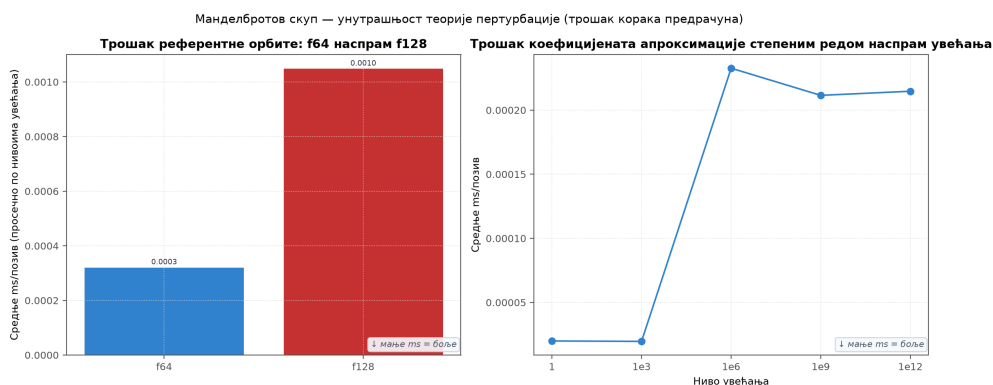
Слика 7.5.1 мери теорију пертурбације, описану у одељку 3.6, наспрам обичног директног кернела, по позадинском решењу, од увећања 1 до 10^{12} . Пертурбација у овом опсегу никада не побеђује, ни на једном од три позадинска решења. Разлика је најизраженија на CUDA-и, где сопствена брза f32 путања наивног кернела рендерује плитки крај серије при знатно преко 1000 Mpix/s, док пертурбација, ограничена на f64 кроз цео опсег, како објашњава одељак 4.2, остаје равна близу дна графика кроз читав опсег. То

директно потврђује сопствени закључак одељка 3.6: ова серија никада не достиже увећање на ком обичан f64 заиста постаје недовољан, тако да је овде пертурбација чист додатни трошак, а не особина исправности каква постаје тек на нивоима увећања изван оних које директни кернел било ког позадинског решења уопште може да представи.



Слика 7.5.1: Теорија пертурбације наспрам обичног директног кернела, по позадинском решењу, кроз дубину увећања, Манделбров токуп.

Слика 7.5.2 објашњава зашто је тај додатни трошак довољно јефтин да буде разумна размена онда када је потребан. Double-double референтна орбита из одељка 3.8 кошта отприлике трипут више него њен обичан f64 парњак по позиву, 0.0010 наспрам 0.0003 ms, а коефицијенти апроксимације степеним редом из одељка 3.7 расту отприлике десетоструко од плитког до дубоког увећања. Обе бројке остају далеко испод једне микросекунде, занемарљиво у односу на кадар размере милисекунде, што је управо разлог зашто одељак 3.8 може да приушти да double-double трошак плати једном по кадру, а не једном по пикселу.



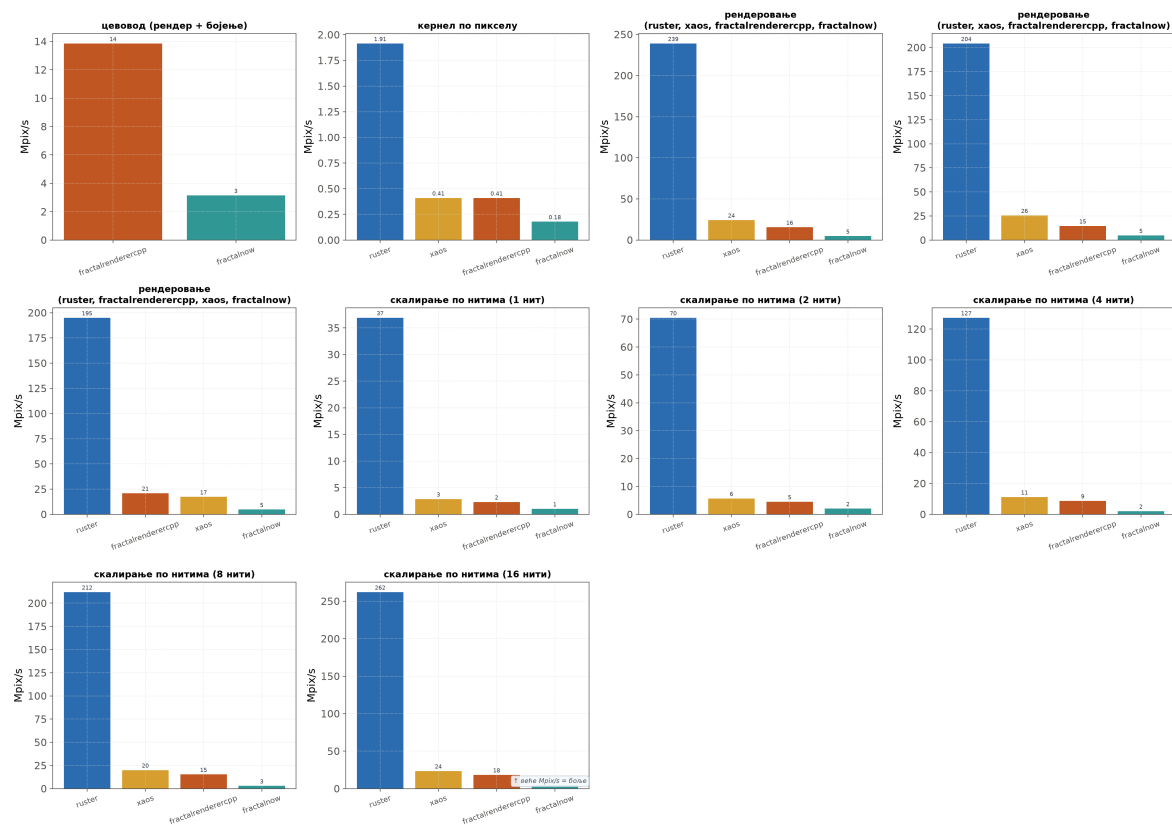
Слика 7.5.2: Трошак предрачуна за пертурбацију, f64 наспрам f128 референтне орбите, и коефицијенти апроксимације степеним редом наспрам увећања.

Теорија пертурбације, дакле, готово ништа не кошта да остане укључена, чак и у опсегу бенчмарка у ком се никада сама не исплати. Наредни одељак се од сопствених позадинских решења raster-а окреће ка томе како се цео рендерер пореди са четири конкурентска пројекта прегледана у одељку 2.2.

7.6 Поређење између пројеката

Слика 7.6.1 ставља `ruster` наспрам три конкурентска рендерера, кроз сваку класу метрика коју овај рад за њих мери: трошак кернела за један пиксел, рендеровање целог кадра на три резолуције, скалирање по нитима од једне до шеснаест, и, где је доступно, цео цевовод, све под протоколом који поставља одељак 6.3.4, уз усклађену резолуцију, број итерација и приказ. `ruster` предњачи на сваком панелу, за отприлике један ред величине над најбржим конкурентом и два реда величине над најспоријим, при сваком испитаном броју нити и свакој резолуцији. Панел цевовода садржи податке само за `FractalRendererC++` и `FractalNow`, пошто се сопствене пречице „нагађања” код Хаос-а и `FractalNow`-а, описане у одељку 6.3.4, примењују на потпуно обојен кадар, а не на `ruster`-ово раздвојено мерење рендеровања и бојења, па њих двоје тамо нису директно упоредиви.

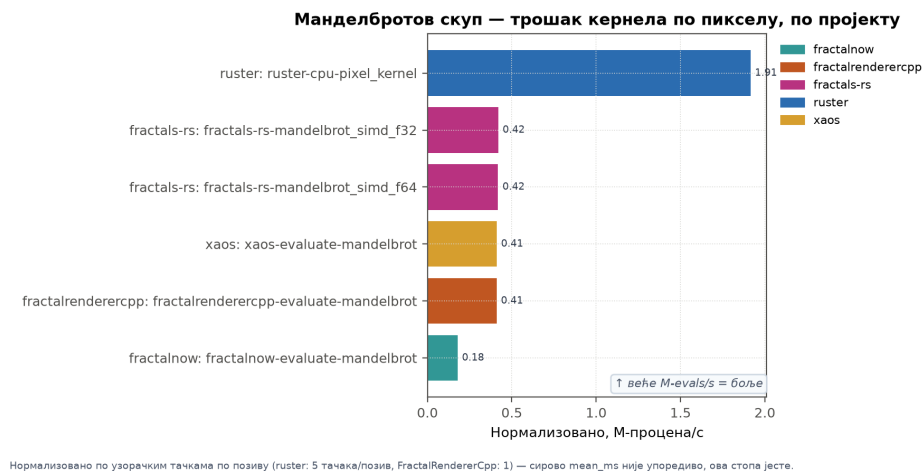
Манделбров скуп — поштено поређење између пројеката (Табела А: усклађен фрактал/резолуција/max_iter/прецизност/алгоритам)



Слика 7.6.1: Поштено поређење између пројеката, усклађен фрактал, резолуција, max_iter, прецизност и класа алгоритма.

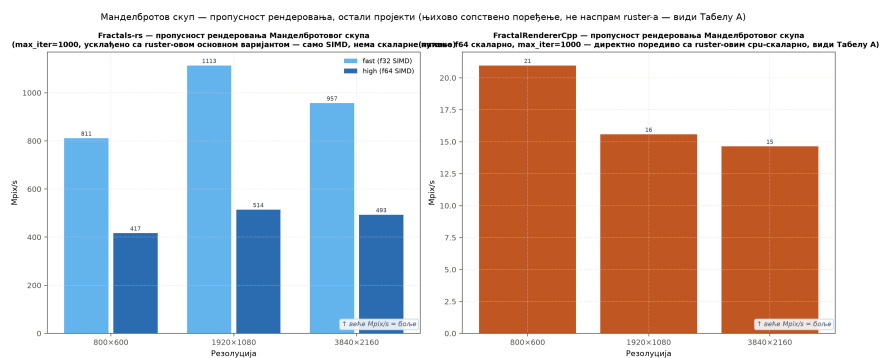
Слика 7.6.2 издваја само кернел по пикселу, нормализован на процене у секунди, тако да `ruster`-ови позиви са више узорачких тачака и позиви других пројеката са једном тачком остају упоредиви, како објашњава одељак 6.1. `ruster`-ов кернел процењује отприлике $4.7\times$ брже од Хаос-овог сировог кернела, са принудно искљученим сопственим праћењем границе у реалном времену, и случајно доспева на готово потпуно исту разлику, отприлике $4.7\times$, над `FractalRendererC++`-овим, чији тест прага излаза поново квадрира $\approx$ уместо да поново користи сопствене посреднике корака ажурирања. `FractalNow`-ов сирови кернел, са искљученом интерполацијом по четворкама, најспорији је од

четири, услед генеричког слоја распоређивања генерисаног макроима, како објашњава одељак 6.3.4, а не због неког конкретnog алгоритамског недостатка.



Слика 7.6.2: Трошак кернела за један пиксел, четири пројекта, Манделбровов скуп, нормализовано на милионе процена у секунди.

Једнонитно поређење са FractalRendererCpp, најпоштеније доступно упаривање, пошто истовремено контролише број нити, прецизност и класу алгоритма, достиже $15.9\times$, 56.2 наспрам 893.0 ms, не захваљујући предности језика, већ зато што ruster-ов кернел већ носи тестове одбацивања за кардиоиду, период 2 и период 3, као и Брентову детекцију циклуса, од којих наивна петља FractalRendererCpp-а нема ниједну. Слика 7.6.3 приказује одговарајућу серију по резолуцији за сам FractalRendererCpp, пропусност равна у оквиру шума кроз све три резолуције, исти потпис „сасвим паралелног” кернела који дефинише одељак 6.4.1. Fractals-rs, покренут у истој сесији као и остала четири, искључиво је SIMD и показује исти равни потпис на обе своје путање прецизности. Његова fast f32 путања ради на 811 до 1113 Mpix/s кроз серију, а high f64 путања на 417 до 514 Mpix/s, свака у оквиру сопственог шума на свакој резолуцији, а не у тренду са величином кадра.



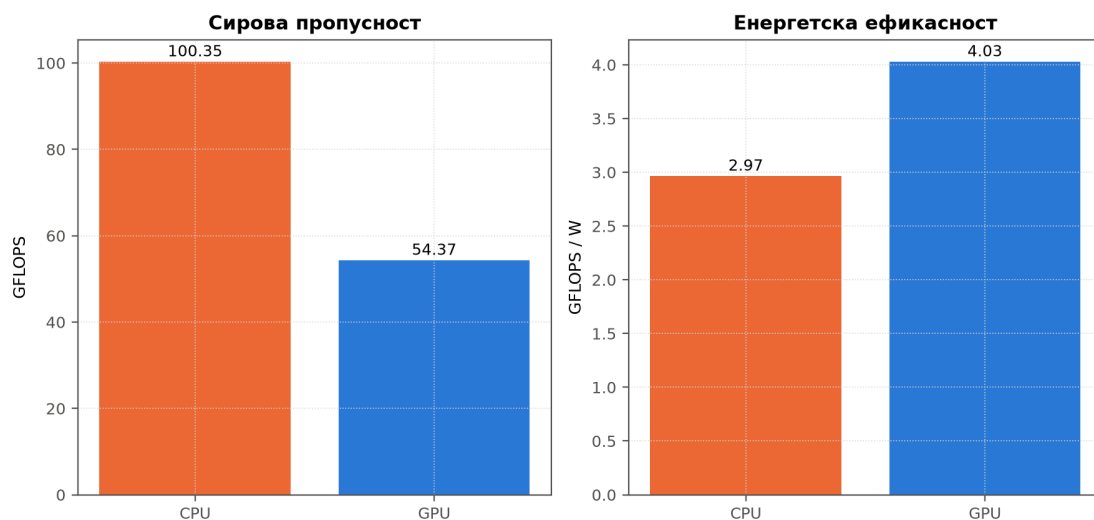
Слика 7.6.3: Сопствена серија по резолуцији за Fractals-rs, f32 наспрам f64 SIMD, и сопствена серија по резолуцији за FractalRendererCpp.

Свако поређење у овом одељку тицало се брзине. Наредно поставља питање на које сама брзина не може да одговори: које је од ruster-ова два сопствена позадинска решења јефтиније за покретање.

7.7 Енергетска ефикасност

Слика 7.7.1 одговара на питање на које само време кадра не може: који је уређај јефтинији за покретање, не само бржи. Графички процесор достиже 54.37 GFLOPS при 13.49 W, наспрам процесорових 100.35 GFLOPS при 33.82 W, тако да процесор побеђује по сировој пропусности, али је графички процесор отприлике 36% ефикаснији по вату, 4.03 наспрам 2.97 GFLOPS/W. Бржи и ефикаснији одговори су на различита питања на овом хардверу, а примена оптимизована за трајање батерије или термички буџет, а не за кашњење, читала би ову слику другачије него што то чини остатак ове главе.

Енергетска ефикасност, процесор наспрам графичког процесора



Слика 7.7.1: Енергетска ефикасност, процесор наспрам графичког процесора, сирови GFLOPS и GFLOPS по вату.

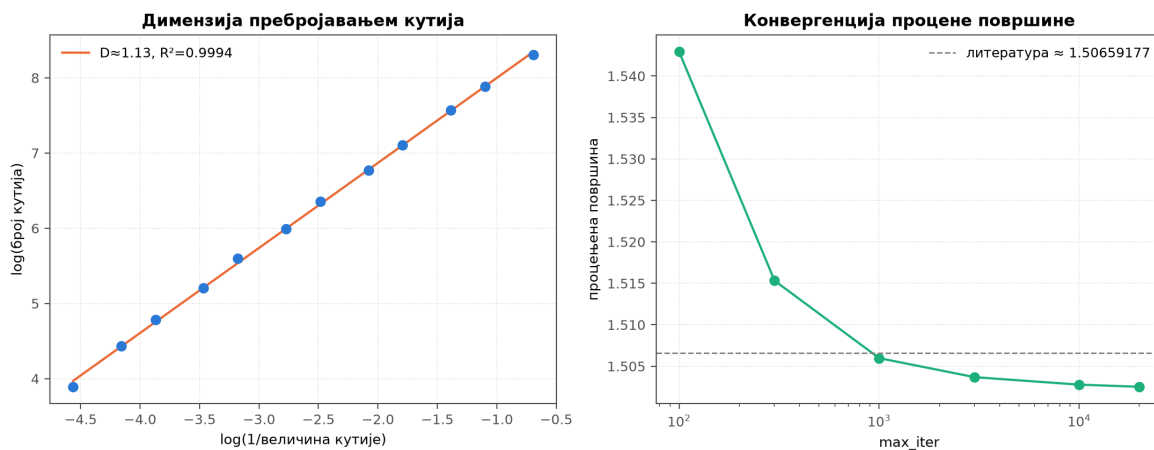
7.8 Нумеричка валидација

Свака ранија секција ове главе питала је колико је рендерер брз. Ова пита да ли је још увек тачан. Главе од 3 до 5 по десетак пута су изнова писале исту „врућу” петљу, од скаларне до SIMD, од процесора до графичког процесора, од директне итерације до пертурбације, а брз рендерер који тихо рачуна погрешан скуп био би гори од спорог, пошто ништа у рендерованој слици не говори колико је погрешна. Овај одељак проверава два својства Манделбрововог скупа која не зависе ни од једног избора имплементације, само од математике из одељка 2.3, и затвара главу потврђујући да је свака до сада измерена оптимизација променила брзину и ништа друго, класа тачности коју одељак 6.4.5 издваја од остатка ове главе.

Леви панел слике 7.8.1 на рендерованој граници фитује димензију пребројавањем кутија (енг. *box counting*), $D \approx 1.10$, уз $R^2 = 0.9907$, знатно испод познате Хаусдорфове димензије границе, тачно 2, вредности коју даје одељак 2.3, очекивана разлика, пошто асимптотска вредност захтева гранични детаљ на размерама финијим него што било који коначан рендер може да достигне. Десни панел варира `max_iter` наспрам процене површине Манделбрововог скупа из литературе, отприлике 1.50659177, и показује да грешка монотонно опада како граница расте, тачно као што предвиђа апрокси-

мација алгоритма бекства из одељка 2.4, пошто премала граница погрешно класификује споро-бежеће граничне пикселе као унутрашње и увећава процену. Ниједан резултат не зависи ни од једне оптимизације у глави 3, оба потичу из обичног скаларног кернела, и оба дају квалитативан одговор какав би дао и другачије имплементиран рендерер, доказ да су оптимизације на другим местима у овом раду промениле брзину, а не тачност.

Нумеричка валидација



Слика 7.8.1: Нумеричка валидација, димензија пребројавањем кутија на рендерованој граници и конвергенција процене површине наспрам границе итерација.

Обе провере погађају тачно тамо где математика каже да треба, независно од било ког резултата брзине у остатку ове главе. Оно што су претходни одељци мерили као пропусност било је, дакле, увек пропусност на тачном одговору.

8. ЗАКЉУЧАК

Овај рад је кренуо да одговори на питање које већина рендерера фрактала заправо никада ни не поставља. Типичан лични рачунар има и вишејезгарни процесор и графички процесор који већи део рендеровања мирују, па зашто се задовољити само једним? Циљ је био изградити рендерер који и путању процесора и путању графичког процесора гура онолико далеко колико свака од њих појединачно може да иде, а затим их повезати распоређивачем који у реалном времену, плочицу по плочицу, одлучује који уређај шта рендерује, уместо да подела буде унапред претпостављена. Полазно очекивање било је да ће путањи процесора бити потребан стваран оптимизациони рад пре него што уопште постане поштен партнер графичком процесору, да ће графички процесор недвосмислено победити када тај рад буде завршен, и да ће распоређивач своју сложеност оправдати само тамо где су два уређаја довољно изједначена да избор ко шта ради заиста нешто значи. Сва три очекивања су се потврдила, мада не увек из разлога претпостављених на почетку, а мерења у глави 7 су запис о томе где су претпоставке биле тачне, а где нису.

Страна процесора се показала важнијом него што се очекивало. Скаларна основна варијанта, проведена кроз паралелну употребу нити у `rayon`-у, а затим кроз `SIMD`, сама по себи достиже убрзање од отприлике $1.9\times$ до $2.4\times$, у зависности од величине кадра, али је већи добитак дошао од јефтиније идеје: провере да ли је доказиво да пиксел припада скупу пре него што се уопште почне итерирати. Тај тест раног изласка, заједно са хватањем циклуса на којима би наивна петља иначе трошила итерације, оно је што је омогућило једној нити процесора да надмаши упоредив `C++` рендерер за фактор од отприлике шеснаест, а да притом ниједна страна не дотакне графички процесор. Не исплати се све. Обилазак Хилбертовом кривом, изабран због својстава локалности кеша, био је спорији од обичног скенирања по редовима на свакој испитаној резолуцији, а теорија пертурбације, изграђена за дубоко увећање, није победила ниједан бенчмарк унутар опсега увећања који је овај рад заиста мерио, јер обична аритметика двоструке тачности у том опсегу једноставно никада није постала недовољна.

Графички процесор је испунио своје обећање сирове пропусности, достигавши отприлике 1389 милиона пиксела у секунди при 4К, далеко испред сваке конфигурације процесора. Оно што се на почетку није очекивало јесте на шта графички процесор заправо троши своје време. Кернели два GPU позадинска решења разликују се за мање од пет процената, а готово цела разлика међу њима долази од копирања готове слике назад на процесор, а не од њеног рачунања. Накнадно бојење слике испоставило се важнијим од самог поређења између два графичка процесора. Мерено са бојењем које и даље ради на процесору за оба позадинска решења, тек тај један корак сам смањио предност графичког процесора над процесором на знатно мање од половине његове предности само у рендеровању. Премештање тог корака на графички процесор испоставило се као једини највећи добитак у перформансама који је овај рад идентификовао, а за `CUDA`-у је рад отишао даље и имплементирао га: кернел за хистограм и читавање из палете на самом уређају, тачан до пиксела у односу на верзију за процесор, 54% до 63% бржи од почетка до краја при плитким и средњим увећањима, тамо где је бојење раније доминирало временом кадра. `wgri` и хетерогени распоређивач и даље боје на

процесору, распоређивач нужно, пошто његов спојени бафер постоји само на страни хоста, тако да тај добитак остаје јасан правац за будући рад на та два пута.

Хибридни распоређивач је део овог рада са највише за испричати. При плитком увећању, где процесор и бржи графички процесор нису блиски по брзини, свака верзија распоређивача, адаптивна или не, губи од простог избора бржег уређаја и његовог самосталног коришћења. Распоређивач своју сложеност оправдава управо у оној једној тачки где два уређаја заиста постају упоредива: на прагу прецизности, где оба прелазе на спорију аритметику веће прецизности и завршавају унутар отприлике 10% брзине један од другог. Тамо, усмеравање скупих плочица ка процесору уместо равномерне поделе кадра побеђује не само бољи уређај сам, него и теоријски најбољи резултат који би слепа подела могла да постигне, за фактор 1.75. Тај резултат је најјаснији доказ који овај рад има за своју централну тврдњу: да распоређивач који сагледа посао пре него што одлучи ко га ради вреди више од оног који само претпоставља. Предност се смањује на дубљим нивоима увећања, остављајући стваран простор који распоређивач још увек не наплаћује.

У поређењу са четири постојећа рендерера отвореног кода, сопствени кернел *ruster*-а био је за отприлике један ред величине бржи од најближег конкурента и за два реда величине бржи од најспоријег, а ниједан део те разлике не потиче од *Rust*-а као језика, пошто је исти рад на раном изласку и детекцији циклуса, који стоји иза бројки за процесор, оно што то објашњава. Ни графички процесор није аутоматски прави одговор. Овде је знатно ефикаснији по вату од процесора, али процесор ипак побеђује по сировој пропусности, тако да „бржи” и „јефтинији за покретање” указују на различите уређаје, у зависности од тога шта се заправо оптимизује. А испод свих бројки брзине лежи тиши, али неопходан резултат. Процена пребројавањем кутија на рендерованој граници и провера конвергенције у односу на познату површину скупа обе погађају тачно тамо где математика предвиђа да треба, што је доказ да је десетак преписивања исте „вруће” петље променило колико је овај рендерер брз, а ништа у вези с тим да ли је тачан.

Два правца произлазе директно из онога што је овај рад измерио, а не из нагађања. Премештање бојења слике на графички процесор промена је највеће вредности која је идентификована, али није изграђена, а подешавање понашања распоређивача при дубоком увећању затворило би стваран, већ квантификован јаз, уместо да захтева нову идеју. Трећи, мањи правац јесте поштено вођење рачуна. Теорија пертурбације никада није тестирана на дубинама увећања за које је заправо и осмишљена, тако да њена стварна исплативост, за разлику од измереног одсуства исплативости у овом раду, остаје отворено питање. Ниједан од ова три правца не захтева преиспитивање нечега у овом раду, само његов наставак.

ЛИТЕРАТУРА

- [1] Michael F. Barnsley. *Fractals Everywhere*. Academic Press, 1988.
- [2] Richard P. Brent. An improved Monte Carlo factorization algorithm. *BIT Numerical Mathematics*, 20(2):176--184, 1980.
- [3] Maxime Collet. Fractals-rs: a Rust fractal explorer. <https://github.com/Maxime-Cllet/Fractals-rs>, 2026.
- [4] T. J. Dekker. A floating-point technique for extending the available precision. *Numerische Mathematik*, 18(3):224--242, 1971.
- [5] Kenneth Falconer. *Fractal Geometry: Mathematical Foundations and Applications*. John Wiley & Sons, 3rd edition, 2014.
- [6] flutomax. Nanobrot: a fork of kalles fraktaler for deep-zoom mandelbrot rendering. <https://github.com/flutomax/nanobrot>, 2020.
- [7] Claude Heiland-Allen. Deep zoom theory and practice. https://mathr.co.uk/blog/2021-05-14_deep_zoom_theory_and_practice.html, 2021.
- [8] David Hilbert. Über die stetige abbildung einer linie auf ein flächenstück. *Mathematische Annalen*, 38(3):459--460, 1891.
- [9] Donald E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, 3rd edition, 1997.
- [10] Victor W. Lee, Changkyu Kim, Jatin Chhugani, Michael Deisher, Daehyun Kim, Anthony D. Nguyen, Nadathur Satish, Mikhail Smelyanskiy, Srinivas Chennupaty, Per Hammarlund, Ronak Singhal, and Pradeep Dubey. Debunking the 100x GPU vs. CPU myth: An evaluation of throughput computing on CPU and GPU. In *Proceedings of the 37th International Symposium on Computer Architecture (ISCA)*, 2010.
- [11] Chi-Keung Luk, Sunpyo Hong, and Hyesoon Kim. Qilin: Exploiting parallelism on heterogeneous multiprocessors with adaptive mapping. In *Proceedings of the 42nd IEEE/ACM International Symposium on Microarchitecture (MICRO-42)*, 2009.
- [12] Benoit B. Mandelbrot. *The Fractal Geometry of Nature*. W. H. Freeman, 1982.
- [13] K. I. Martin. Superfractalthing maths: Theory and practice of perturbation in complex dynamics. https://superfractalthing.co.nf/sft_maths.pdf, 2013.
- [14] Michał Męciński. Fraqtive: a SIMD and multi-core mandelbrot-family fractal generator. <https://github.com/mimecorg/fraqtive>, 2023.
- [15] John Milnor. *Dynamics in One Complex Variable*. Annals of Mathematics Studies. Princeton University Press, 3rd edition, 2006.

- [16] Guy M. Morton. A computer oriented geodetic data base and a new technique in file sequencing. Technical report, IBM Ltd., Ottawa, Canada, 1966.
- [17] Takeshi Ogita, Siegfried M. Rump, and Shin'ichi Oishi. Accurate sum and dot product. *SIAM Journal on Scientific Computing*, 26(6):1955--1988, 2005.
- [18] Marc Pégon. Fractalnow: an open-source, multi-threaded fractal generator. <http://fractalnow.sourceforge.net/>, 2017.
- [19] Heinz-Otto Peitgen and Dietmar Saupe, editors. *The Science of Fractal Images*. Springer-Verlag, New York, 1988.
- [20] Mark Seufert. Fractalrenderercpp: a C++ fractal rendering library. <https://github.com/MarkSeufert/FractalRendererCpp>, 2021.
- [21] XaoS Project. XaoS: a real-time interactive fractal zoomer. <https://github.com/xaos-project/XaoS>, 2026.

БИОГРАФИЈА

Тамаш Пап рођен је 30.01.2004. године у Сенти. Завршио је основну школу „Кокаи Имре” у Темерину као вуковац. Похађао је Гимназију за талентоване ученике „Бољаи” у Сенти, природно-математички смер. По завршетку средње школе уписао је смер Рачунарство и аутоматика на Факултету техничких наука у Новом Саду, где је студирао од 2022. до 2026. године.



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР :	
Идентификациони број, ИБР :	
Тип документације, ТД :	монографска публикација
Тип записа, ТЗ :	текстуални штампани документ
Врста рада, ВР :	дипломски рад
Аутор, АУ :	Тамаш Пап
Ментор, МН :	др Вељко Петровић, доцент
Наслов рада, НР :	Фрактал рендеровање са хибридном паралелизацијом
Језик публикације, ЈП :	српски
Језик извода, ЈИ :	српски / енглески
Земља публикавања, ЗП :	Република Србија
Уже географско подручје, УГП :	АП Војводина
Година, ГО :	2026.
Издавач, ИЗ :	ауторски репринт
Место и адреса, МА :	Нови Сад, Факултет техничких наука, Трг Доситеја Обрадовића 6
Физички опис рада, ФО : (поглавља/страна/ цитата/табела/слика/графика/прилога)	8 / 56 / 21 / 0 / 12 / 17 / 0
Научна област, НО :	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД :	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО :	фрактали, Манделбровов скуп, паралелно рачунарство, GPU рачунарство, хибридно распоређивање
УДК	
Чува се, ЧУ :	Библиотека Факултета техничких наука, Трг Доситеја Обрадовића 6, Нови Сад
Важна напомена, ВН :	
Извод, ИЗ :	Изложене су теоретске основе алгоритма бекства и имплементиран је хибридни рендерер Манделбрововог скупа који комбинује оптимизовани процесор и графичку картицу, уз распоређивач који адаптивно дели посао између њих. Анализа показује убрзање у односу на најбоље самостално решење.
Датум прихватања теме, ДП :	
Датум одбране, ДО :	
Чланови комисије, КО :	Председник: проф др Дину Драган, редовни професор
	Члан: проф др Дину Драган, редовни професор
	Члан: проф др Душан Гајућ, редовни професор
	Члан, ментор: др Вељко Петровић, доцент
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	monographic publication
Type of record, TR :	textual printed document
Contents code, CC :	bachelor thesis
Author, AU :	Tamaš Pap
Mentor, MN :	Veljko Petrović, Assistant Professor, PhD
Title, TI :	Fractal rendering with hybrid parallelism
Language of text, LT :	serbian
Language of abstract, LA :	serbian / english
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	author's reprint
Publication place, PP :	Novi Sad, Faculty of Technical Sciences, Trg Dositeja Obradovića 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	8 / 56 / 21 / 0 / 12 / 17 / 0
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied Computer Science and Informatics
Subject/Key words, S/KW :	fractals, Mandelbrot set, parallel computing, GPU computing, hybrid scheduling
UC	
Holding data, HD :	Library of the Faculty of Technical Sciences, Trg Dositeja Obradovića 6, Novi Sad
Note, N :	
Abstract, AB :	The theoretical foundations of the escape-time algorithm are presented, and a hybrid Mandelbrot set renderer is implemented, combining an optimized CPU path with a GPU path, along with a scheduler that adaptively divides work between them. The analysis shows a speedup over the best standalone solution.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	
Defended Board, DB :	
President:	Dinu Dragan, Full Professor, PhD
Member:	Dinu Dragan, Full Professor, PhD
Member:	Dušan Gajić, Full Professor, PhD
Member, Mentor:	Veljko Petrović, Assistant Professor, PhD
	Mentor's sign